

THESIS WORK

Shayan Ghiaseddin
2026

Corvinus University of Budapest
Institute of Data Analytics and Information
Systems

Detecting Open Source Library Usage and License Disclosure Gaps in Android APKs

Shayan Ghiaseddin
Business Informatics MSc Programme
2026

Thesis Supervisor: Dr. Johannes Wachs
Associate Professor

Contents

Summary	6
1 Introduction	7
1.1 Problem Statement	7
1.2 Research Objectives and Scope	7
1.3 Research Questions	8
1.4 Structure of the Thesis	8
1.5 Personal Motivation and Acknowledgements	8
2 Theoretical Background	9
2.1 Modern Software Development and Dependency Ecosystem	9
2.2 Open Source Libraries and Their Benefits	10
2.3 Dependency Networks in Modern Software Systems	11
2.4 Security Incidents Caused by Vulnerable Dependencies	13
3 Industry Relevance and Sensitive Sectors	15
3.1 Why Industry Context Matters	15
3.2 Financial Services	15
3.3 Healthcare and Health Technology	15
3.4 Retail and eCommerce	16
3.5 Internet, Mobile, and AI-Intensive Software	16
4 Literature Review	17
4.1 Review-Based Studies Relevant to the Thesis	17
4.2 Main Research Questions, Methods, and Findings in the Review Corpus	17
4.3 Why This Topic Attracts Research Attention	20
4.4 Research Gaps Identified in Prior Reviews	20
5 Methodology	22
5.1 Research Design	22
5.2 Research Workflow	22
5.3 APK Dataset Selection	23
5.4 Dependency Extraction Approach	23
5.5 Fingerprint-Based Identification	23
5.6 Data Aggregation and Analysis	23
5.7 Repository Metadata and Health Metric	24
5.8 Methodological Limitations	24

6	System Design and Pipeline	25
6.1	Overview	25
6.2	Input Data Sources	25
6.3	APK Acquisition and Decoding	25
6.4	License Extraction	27
6.5	Repository Collection	27
6.6	Fingerprint Generation	27
6.6.1	Illustrative Example of Open-Source Library Detection	27
6.7	Library Matching in APK Files	28
6.8	Aggregation and Reporting	28
6.9	Computational Efficiency Techniques	29
7	Data Analysis	30
7.1	Generated Dataset	30
7.1.1	Corpus Scaling Strategy	30
7.1.2	Evaluated Metrics	31
7.1.3	Observed Trends	31
7.1.4	Final Dataset Selection	34
7.2	GitHub-Based Repository Health Score	34
7.2.1	Adopted Health-Score Model	34
7.2.2	Application to Repositories Detected in the APK Corpus . . .	35
7.2.3	Repository Health-Score Results	36
8	Validation	38
8.1	Validation Strategy	38
8.2	Open-Source Control Set	38
8.3	Source-Code Library Extraction Methods	40
8.4	Matching APK Findings Against Source-Code Findings	40
8.5	Validation Results on the F-Droid Control Set	41
8.6	Limitations of Validation	41
9	Results and Discussion	43
9.1	Findings	43
9.1.1	Summary of Key Performance Indicators (KPIs)	43
9.1.2	Distribution of Libraries Across APKs	44
9.1.3	Medium-Health Level Open Source Library Usage in APKs . .	45
9.2	Interpretation of Results	46
9.3	Implications for Developers	47
9.4	Implications for End Users	48
10	Conclusion	49
10.1	Answer to Research Questions	49
10.2	Future Work	50
	References	51

Appendix	55
.1 Wang and Wu OSS Repository Health Model	55
.1.1 Methodological Structure	55
.1.2 Data and Preprocessing	55
.1.3 Weighting Method	55
.1.4 Health-Score Calculation	56
.2 Extra Figures and Tables	56

List of Figures

6.1	Overview of the data acquisition and analysis pipeline	26
6.2	Sample evidence of Glide library usage in a decoded APK	28
7.1	Distribution of evidence rows per APK	32
7.2	Growth of unique libraries discovered as corpus size increases	32
7.3	Share of cited library records also confirmed by fingerprint evidence across corpus sizes	33
7.4	Ratio of cited vs. uncited libraries across corpus sizes	33
7.5	Top 10 repositories health score in the APK corpus	36
7.6	Distribution of repositories across health levels in the APK corpus . .	37
8.1	Distribution of detected open source library use per APK in the F-Droid validation set	39
9.1	Distribution of cited versus uncited libraries	44
9.2	Distribution of evidences of open source library use per APK	45
9.3	Top 10 APKs by medium-health library usage intensity	46
9.4	Top 10 APKs by uncited medium-health library usage intensity . . .	47
1	The regular expressions to capture fingerprints in varius languages such as Java (code in python)	56
2	Distribution of fingerprint types used to detect library evidence by application category in the final APK corpus	57
3	Total evidences of open source library use as the APK corpus size increases	57
4	Detection origin of cited library records, comparing fingerprint evi- dence and license-exposure evidence by application category	58
5	BudapestGO in-app license screen generated through Sentry, a helper tool used in the CI/CD stack to audit runtime external libraries and export their licenses. Screenshot address: Menu, Information and help, License.	59
6	Histogram of evidences of open source library use per APK in the final corpus	60

List of Tables

4.1	Main research questions in the underlying corpus of studies on open-source and third-party libraries in Android APKs	18
7.1	APK count by Google Play category in the final corpus	34
7.2	Repository health factors adopted from Wang and Wu’s OSS health model	35
8.1	Distribution of match types in the F-Droid validation set	41
8.2	Validation metrics using source-code analysis as approximate ground truth	41
1	Summary report for the BudapestGO app	58
2	List of apps for manual license investigation.	61
3	Merged summary KPIs for the 699 APK corpus	62

Summary

Modern Android applications are built from many external open-source libraries. These libraries help developers build apps faster, but they also create a visibility problem: users, auditors, and sometimes even developers may not know which third-party components are actually embedded inside an app. This matters because external libraries can contain vulnerabilities, become unmaintained, or create license obligations. If a library is used but not declared, its risk is difficult to notice and manage.

This thesis addresses that problem by developing an automated pipeline for detecting open-source library usage in compiled Android APK files. The pipeline collects APKs, decodes them, extracts license information, builds fingerprints from public source-code repositories, and matches those fingerprints against decoded application code. This makes it possible to compare what an application declares in its license information with what is actually detected in the code.

The final analysis is based on a corpus of 699 Android APKs from sensitive categories such as finance, health, communication, and navigation. The results show that Android applications rely heavily on external libraries, with an average of 154.84 detected open source libraries per APK. However, 44.61% of detected libraries were not cited in license declarations. After investigating repository of open source libraries, the analysis also shows that 89 applications depend on at least one medium-health libraries, including 34 cases where those libraries are uncited. The fingerprint-based method proved effective, confirming most library usage through code evidence and revealing dependencies that license information alone would miss.

Overall, the thesis shows that the main risk is not simply the use of open-source libraries, but the gap between actual library usage and visible disclosure. The proposed pipeline provides a practical way to measure this gap and supports better dependency transparency in Android applications. In practical use, the pipeline can analyze a given APK and report which open-source libraries it contains and how healthy those libraries appear to be.

Chapter 1

Introduction

1.1 Problem Statement

Modern software development heavily relies on open source libraries and external dependencies. Android applications are no exception, often incorporating numerous third-party components whose presence is not always documented in a standardized way. Most of the times, the original source code of mobile applications is not publicly available; for Android applications, researchers usually have access only to compiled APK files. This makes it difficult to determine which open source libraries are actually used by widely distributed consumer applications, and how deeply applications depend on external code. Existing tools for detecting library usage inside APK files are limited. Therefore, there is a need for a scalable method to extract, identify, and analyze open source software references from Android applications and report potential risks.

1.2 Research Objectives and Scope

In this thesis, the terms *external libraries*, *open source libraries*, *open source software (OSS)*, and *libraries* refer to reusable third-party software components included in an Android application to provide specific functionality.

The objective of this thesis is to develop an automated and scalable pipeline capable of extracting references to open source libraries from Android APK files. The research aims to build a dataset of commonly used applications, detect declared and undeclared library dependencies, and analyze the extent of these dependencies. Another goal is to evaluate how reliably library usage can be identified through license declarations, package fingerprints, and repository metadata, and to assess the feasibility of large-scale dependency analysis in mobile applications. The thesis also aims to identify applications with the highest intensity of uncited unhealthy open source library usage.

The study is limited to Android APK files and does not cover iOS applications or other mobile platforms. For the main APK corpus, decoded evidence is not validated against original application source code, because that source code is generally unavailable. However, for the separate validation dataset, I intentionally selected applications with available source code and performed this comparison, as discussed in Chapter 8. Finally, the thesis does not audit applications at runtime; library usage is evaluated through static evidence extracted from decoded APKs.

1.3 Research Questions

The research is guided by the following questions:

- (RQ1) Is it feasible to detect external libraries in Android APK files?
- (RQ2) How reliable are license declarations and package fingerprints for identifying library dependencies?
- (RQ3) How many open source libraries, and on what health level, are used in a sample of Android applications?
- (RQ4) Which Android applications show the highest dependency on low-health or medium-health open-source libraries?

1.4 Structure of the Thesis

We first review open-source dependency risks in Android software, focusing on prior work about Android third-party library detection, software supply-chain security, and open-source license management. We then move to the pipeline I developed to collect APKs, extract library evidence, and build the dataset. Finally, the generated dataset is analyzed to compare detected library usage with license disclosure and to identify applications with higher uncited unhealthy-library exposure.

1.5 Personal Motivation and Acknowledgements

I chose this topic because I enjoy technical challenges. Detecting dependencies from the source code of Android applications would be relatively straightforward, but APK files are compiled final products where the original project structure is no longer directly visible. This makes dependency detection more difficult and makes the problem more interesting from both a technical and research perspective.

I would like to thank my supervisor, Dr. Johannes Wachs, for involving me in this research topic and for his guidance, support, and enthusiasm throughout the work.

Also, I would like to thank my wife, Shahrzad to support me in this journey.

Chapter 2

Theoretical Background

2.1 Modern Software Development and Dependency Ecosystem

Modern software development has evolved from monolithic, self-contained programs toward highly modular systems built from reusable components, libraries, and services developed by third parties. This shift has been driven by increasing software complexity, economic pressure to improve productivity, and the emergence of collaborative development ecosystems.

In the early decades of computing, software was typically written as monolithic programs tailored for specific hardware systems. Programs were often developed from scratch, with little separation between components, and reuse was limited to informal sharing among research institutions and user groups (Ojha & Rao, 2014). As software systems grew larger in the 1960s, developers encountered what became known as the *software crisis*, characterized by cost overruns, unreliable systems, and difficulty managing complexity. The 1968 NATO Software Engineering Conference highlighted the need for disciplined engineering practices and emphasized modular design as a way to divide large problems into smaller, manageable units (Randell, 1979).

During the same period, the idea of systematic software reuse emerged. McIlroy proposed that software could be assembled from standardized modules rather than built entirely from scratch, introducing the concept of catalogs of reusable components. In parallel, early *software factory* concepts suggested that development should follow standardized processes, reusable parts, and controlled environments to improve productivity and quality. This terminology became especially influential in Japan during the mid-1970s and 1980s, where software production was increasingly compared with industrial manufacturing (Cusumano, 1989). These ideas reflected the growing recognition that modularity was not only a technical choice but also an economic necessity.

In the 1970s, structured programming and principles such as *coupling and cohesion* formalized the concept of modular design. High-level programming languages and improved compiler technology enabled independent modules to be developed and combined into larger systems, supporting parallel development and easier maintenance (Newman, 2019). At the same time, software began to be treated as a commercial product rather than an inseparable part of hardware, further encouraging the creation of reusable libraries and third-party components.

The 1980s marked an important milestone with the rise of the *free software* movement, later known as open source software. Projects such as GNU promoted the idea that software should be freely shared, modified, and redistributed, supported by licenses designed to preserve openness. This created a growing pool of reusable tools, compilers, and libraries that developers could incorporate into their own systems (Ojha & Rao, 2014). Object-oriented programming also became widespread during this period, encouraging encapsulation and reuse through class libraries and reusable frameworks (del Águila et al., 2020).

In the 1990s, open-source development moved from a niche practice to a widely adopted model. Large collaborative projects such as Linux demonstrated that complex systems could be built from many independently developed modules contributed by distributed communities. The emergence of public code repositories and standardized open-source licenses made it easier for developers to discover, integrate, and reuse third-party libraries (Ojha & Rao, 2014). Component-based development and reusable frameworks became common, further reinforcing the modular approach (del Águila et al., 2020).

During the 2000s, open source became commercially mainstream. Organizations increasingly relied on external libraries instead of developing all functionality in-house, reducing development costs and accelerating delivery through improved collaboration platforms and mature licensing models. At the architectural level, service-oriented architecture extended modularity beyond source code by allowing systems to be composed of independent services communicating over networks (del Águila et al., 2020; Fitzgerald, 2006).

In the 2010s, *microservices architecture* and *cloud computing* accelerated the shift toward dependency-based development. Applications began to consist of many small, independently deployable components, often combined with third-party libraries and external services. Modern software projects frequently contain relatively little original code, acting instead as orchestration layers around large dependency graphs maintained by global developer communities (del Águila et al., 2020; Newman, 2019).

As a result of this historical evolution, contemporary software development depends heavily on external libraries and open-source components. While this modular ecosystem enables rapid innovation, it also creates challenges related to dependency management, transparency, and security, especially when the exact set of third-party components used by an application is not clearly documented. These challenges motivate the need for methods capable of identifying and analyzing software dependencies. But first, we focus on the benefits of open-source libraries.

2.2 Open Source Libraries and Their Benefits

Open-source libraries are central to modern software development because they provide reusable functionality that can be inspected, modified, and improved collaboratively. Their main benefit is not only code availability, but also collective review: many developers and users can test, improve, and maintain the same components, which can support innovation, faster defect discovery, and software quality comparable to proprietary alternatives (O'Reilly, 1999; Weber, 2000).

Another important benefit of using open-source libraries is cost efficiency and faster development. Instead of building all functionality internally, organizations can integrate proven components, share the cost of common infrastructure, and

focus internal resources on competitive features. This reduces development time, lowers financial costs, and improves productivity through repeated reuse across projects (Ajila & Wu, 2007; Chesbrough, 2023; Lerner & Tirole, 2005). At the industry level, the economic value of open-source software is substantial, as replacing open-source components with proprietary alternatives would require significantly higher investment and development effort (Hoffmann et al., 2024).

Open-source libraries also support knowledge sharing across the software ecosystem. Because source code is available, improvements created for one project can be inspected, adapted, and reused by others, leading to faster diffusion of technical knowledge, professional learning, and reduced duplicated effort across organizations (Chesbrough, 2023; Lerner & Tirole, 2002).

Because of these advantages, applications today are rarely developed entirely from scratch; instead, they are assembled from numerous external components maintained by global communities. While this approach enables rapid development and innovation, it also increases the complexity of dependency relationships.

2.3 Dependency Networks in Modern Software Systems

Modern software development has become heavily reliant on open-source libraries and modules, creating extensive dependency networks that span entire ecosystems. A 2021 audit of over 947 commercial codebases found that 98% contained open-source libraries (Duck, 2026). Each application-level dependency can itself have further requirements, forming a direct and transitive dependency graph. In other words, *software supply chains* now consist of layers of upstream libraries and their downstream dependents, interwoven networks of packages maintained by vast communities.

In an open-source dependency network, developers declare direct dependencies on certain library packages, and each of those libraries may bring in its own dependencies (transitive dependencies). Package managers, as development tools, automatically resolve and fetch this full transitive closure of the dependency graph to build the project (Boldi & Gousios, 2020). As a result, a project that directly uses only a handful of libraries might end up including dozens or hundreds of indirect libraries through recursive dependency chains. This deep nesting of components means that modern applications implicitly trust not only the libraries they explicitly import but also all the dependencies of those dependencies. The dependency graph thus often spans many levels, and managing it requires considering not just individual packages but the entire tree of interconnected software components. Over time, these open-source dependency networks have grown immense in scale and depth. For example, the npm repository for Node.js now hosts over 5.5 million packages (“Libraries.io - Security & Maintenance Data for Open Source Software”, 2026), and other ecosystems (Maven, PyPI, RubyGems, etc.) have similarly exploded in size. The typical software project today relies on dozens of third-party libraries: according to a corporate report, the average number of open-source components per application was 1,180, and 64% of open-source components in a typical codebase were transitive dependencies pulled in automatically by direct dependencies (Duck, 2026).

Critically, the dependency networks are not only large but also highly interconnected. Boldi and Gousios (2020) observe that ecosystems have become extremely

fragile, with certain core libraries serving as hubs that many others rely on. In the RubyGems ecosystem, for instance, a single popular library was found to appear in the dependency trees of over 400,000 other packages – meaning that if this one library were removed, about 40% of the entire ecosystem would cease to work (Zerouali et al., 2022). This example shows how a small change in a widely used component can have a cascading effect across countless applications. The overall number of issues scales with this growth as well: according to the 2026 Black Duck OSSRA report, audited codebases contained an average of 581 total vulnerabilities but only 237 average unique vulnerabilities, showing that many vulnerabilities appear multiple times across the same application because of duplicated and repeated dependencies in complex dependency trees (Duck, 2026).

One prevalent problem arising in these sprawling networks is *dependency bloat*. This refers to libraries that are included in a project’s dependency tree but are never actually used by the project at runtime. Bloated dependencies needlessly increase software footprint and can even introduce additional risk or maintenance work without any benefit. Empirical analysis of the Maven Central (Java) ecosystem shows that bloated dependencies are common and tend to persist over time: in one longitudinal study of 435 Java projects, the number of bloated (unused) dependencies steadily increased with each release, and fully 89% of the dependencies identified as bloated in a given project version remained bloated in all subsequent versions of that project (Soto-Valero et al., 2021). In other words, once a library is added and turns out to be unnecessary, developers rarely remove it. In addition, Soto-Valero et al. (2021) found that about 22% of the dependency update operations performed by developers were devoted to updating these unused libraries. Such effort is essentially wasted, since those libraries did not need to be there in the first place. Dependency bloat shows how deep dependency networks can make the overall software stack heavier and more difficult to manage.

Another critical issue introduced by deep and complex dependency networks is the *propagation of vulnerabilities* through the dependency tree. When a security flaw is discovered in a library, any application that directly or transitively depends on that library may be exposed to the vulnerability. As Boldi and Gousios (2020) point out, a bug or vulnerability in a single low-level library can affect a huge number of other libraries and applications that rely on it, whether directly or via transitive inclusion. Düsing and Hermann (2022) similarly report that vulnerabilities often influence downstream software through long chains of dependencies, such that a single vulnerable library version may end up impacting thousands of other artifacts that include it transitively. In effect, the dependency network acts as a wide distribution pipeline for exploits: a weakness in one component can radiate outward to many projects.

Managing and mitigating these security risks has proven to be extremely challenging as dependency networks deepen. Developers and organizations frequently struggle to keep their dependencies up-to-date with the latest security patches. Düsing and Hermann (2022) note that, despite the availability of automated tools (such as dependency bots and vulnerability scanners), many developers find it difficult in practice to stay on top of upgrading vulnerable libraries. The result is that known vulnerabilities can linger in software for long periods before they are addressed. A comprehensive multi-language study by Kumar et al. (2024) observed that once a serious vulnerability is introduced via a dependency, it tends to persist for over a

year on average in the affected projects before being fixed. This lengthy window of exposure underscores the slow adoption of security patches in dependency networks. Even when a fix has been released upstream, downstream projects do not always incorporate it promptly. Research on the npm and RubyGems ecosystems found that roughly 33% of dependency vulnerabilities in GitHub projects could be resolved by simply upgrading to a newer patched version within the same major release of the library – yet those projects had not done so (Zerouali et al., 2022). When a new CVE (Common Vulnerabilities and Exposures) is announced in a popular library, maintainers of downstream projects must determine whether their software is impacted – a non-trivial task when the vulnerable code may be buried several layers deep in a transitive dependency. In summary, the deeper and more entangled the dependency network, the harder it becomes to monitor and manage security across all the included components.

2.4 Security Incidents Caused by Vulnerable Dependencies

Historical evidence shows that some of the most severe cybersecurity incidents of the last decade were caused by flaws in open-source dependencies rather than in the application code itself (Ponta et al., 2020; Sonatype, 2026). The Heartbleed vulnerability in 2014 affected OpenSSL, a cryptographic library used by a large share of web servers worldwide. A buffer over-read flaw allowed attackers to access sensitive memory contents, including private keys and credentials, forcing emergency updates across millions of systems (Sonatype, 2026). In the same year, the Shellshock vulnerability in the GNU Bash shell enabled remote code execution on Unix-based systems, demonstrating that even low-level open-source utilities can expose a large attack surface when they are widely deployed (Sonatype, 2026).

Dependency-related incidents continued in later years. In 2017, attackers exploited an unpatched vulnerability in the Apache Struts web framework during the Equifax data breach, gaining access to the personal data of approximately 147 million individuals. The flaw had been publicly disclosed months earlier, but the affected systems were not updated in time, illustrating the risks of slow patch adoption in dependency networks (Cordey, 2023; Sonatype, 2026). The 2018 event-stream incident in the npm ecosystem demonstrated that malicious code could be inserted directly into a dependency. A compromised version of a widely used JavaScript package included hidden code designed to steal cryptocurrency wallet credentials, demonstrating that attacks may also originate from manipulated updates rather than accidental bugs (“Npm Blog Archive”, 2018).

More recent incidents highlight the growing scale of the problem. Discovered in 2021, the Log4Shell vulnerability in the Apache Log4j logging library enabled remote code execution through specially crafted log messages and affected a vast number of enterprise and cloud applications. Because Log4j was included as a dependency in countless projects, organizations worldwide had to audit their systems to determine whether the vulnerable library was present, including cases where it was only included transitively (Ponta et al., 2020; Yan et al., 2021). In the JavaScript ecosystem, a severe prototype-pollution flaw in Lodash was found to potentially expose over 4 million GitHub projects through dependency graphs (C. Liu et al., 2022). In 2022,

the Spring4Shell vulnerability again showed how flaws in widely used frameworks can be exploited to deploy malware on servers that indirectly depend on the affected library (“CVE-2022-22965 Analyzing the Exploitation of Spring4Shell Vulnerability in Weaponizing and Executing the Mirai Botnet Malware”, 2022).

A different type of dependency risk was demonstrated by the XZ Utils backdoor discovered in 2024. In this case, malicious code was intentionally introduced into the source of a widely used compression library by a contributor who had gained maintainer trust. Unlike ordinary vulnerabilities, this was not an accidental coding flaw but a deliberate supply-chain compromise that could have allowed remote access if it had reached stable releases (Freire, 2024). This incident emphasized the difficulty of securing open-source ecosystems where many projects depend on code maintained by external contributors.

Available statistics indicate that dependency-related security problems are increasing over time. CVE disclosures show a strong upward trend: reported vulnerabilities grew by 463% between 2013 and 2023, and by the first half of 2025 more than 21,500 CVEs had already been disclosed, roughly 133 per day, with about 38% rated high or critical. Malicious package activity has also accelerated: Sonatype reported 704,102 malicious open-source packages discovered since 2019, including more than 512,000 new malicious packages since November 2023, a 156% year-over-year increase (“Open Source Malware Index Q3 2025”, 2025; “Vulnerabilities Statistics 2025”, 2025). These figures show why identifying the libraries embedded in compiled Android APK files is important: dependency risks can propagate widely, and their impact becomes especially serious in sectors where software handles sensitive data, regulated processes, or critical services.

Chapter 3

Industry Relevance and Sensitive Sectors

3.1 Why Industry Context Matters

The risks created by external libraries are not distributed evenly across all software domains. Their impact depends on the sensitivity of the data processed, regulatory oversight, third-party integrations, and the organization’s ability to monitor dependencies over time. Although open-source usage is now nearly universal in commercial software, some sectors face stronger pressure because they combine deep dependency trees, strict compliance requirements, many interconnected applications, and serious consequences in the event of failure. In these sectors, incomplete visibility into external components can create operational, legal, and security risks that are more serious than in less regulated domains (Duck, 2026; Snyk, 2024; Sonatype, 2026).

3.2 Financial Services

Financial services are one of the most clearly sensitive sectors for dependence on external libraries. According to the 2026 OSSRA report, Financial Services and FinTech codebases show very high concentrations of high-risk vulnerabilities, reaching 92%, while the same report also lists this sector among the industries with the highest license-conflict rates at 72% (Duck, 2026). This pattern is linked to large and complex codebases, many integration points, and regulatory requirements that can slow updates and remediation (Duck, 2026). From a governance perspective, Sonatype notes that regulated industries, especially financial services, are early adopters of Software Bills of Materials (SBOMs), while the Digital Operational Resilience Act (DORA) requires financial institutions and ICT providers to strengthen oversight of software supply-chain risk (Sonatype, 2026). Therefore, external libraries in financial software are not only a technical dependency issue, but also part of compliance and audit responsibility (Duck, 2026; Sonatype, 2026).

3.3 Healthcare and Health Technology

Healthcare, health technology, and life sciences also stand out as highly sensitive sectors. In the OSSRA industry breakdown, Healthcare, Health Tech, and Life

Sciences codebases show high levels of serious vulnerability exposure, with 88% of audited codebases containing high-risk vulnerabilities (Duck, 2026). The same report links this to regulatory constraints, complex integrations such as electronic health record (EHR) and insurance systems, and a cautious change culture that can slow updates or modernization (Duck, 2026). Because healthcare software often processes sensitive regulated information, outdated or poorly governed external libraries can create both security and compliance risks, making software assurance, transparency, and supply-chain documentation especially important (Duck, 2026; Sonatype, 2026).

3.4 Retail and eCommerce

Retail and eCommerce are also sensitive because they combine customer-facing systems, payment integrations, analytics tools, and high-volume transactions. The OSSRA report shows that this sector has one of the highest concentrations of high-risk vulnerabilities, at 93% of audited codebases, and links this exposure to complex applications, extensive third-party integrations, and PCI-related requirements that can slow remediation (Duck, 2026). As a result, external library risks in retail systems can directly affect customer operations, payment security, and business continuity (Duck, 2026; Sonatype, 2026).

3.5 Internet, Mobile, and AI-Intensive Software

Internet-facing and mobile-oriented software is another major area of relevance. The OSSRA report lists Internet and Mobile Apps at 78% for high-risk vulnerability prevalence and 71% for license conflicts, indicating that dependency-heavy application stacks in this domain are especially exposed to both security and compliance problems (Duck, 2026). The same report also identifies Big Data, AI, BI, and Machine Learning among the industries with both high vulnerability prevalence, at 88%, and the highest license-conflict rate, at 78% (Duck, 2026).

These findings align with broader supply-chain concerns. Sonatype argues that software supply chains have reached machine scale, that package ecosystems are under structural pressure, and that AI-assisted development can amplify dependency churn, version-selection mistakes, and malware exposure when decisions are made without live intelligence and governance controls (Sonatype, 2026). Black Duck similarly reports that AI-generated code introduces additional licensing and IP uncertainty, while Snyk shows that many organizations remain concerned that AI coding tools can introduce both security vulnerabilities and open-source licensing problems into their applications (Duck, 2026; Snyk, 2024; Sonatype, 2026).

Chapter 4

Literature Review

4.1 Review-Based Studies Relevant to the Thesis

This literature review focuses on review papers and survey-based studies that synthesize prior research on Android third-party libraries, open-source license management, software supply chain security, Android vulnerability detection, Android malware analysis, and dynamic Android security analysis. I used these studies because the topic sits at the intersection of several research streams, and review papers provide a structured overview of the dominant questions, methods, findings, and unresolved gaps in those streams.

The most central review for this thesis is the systematic literature review by Zhan et al. (2022), because it directly maps research on third-party libraries in Android applications. Also, library identification in APKs is strongly connected to software supply chain security, because embedded open-source components can introduce vulnerabilities, privacy risks, license obligations, and maintenance problems into downstream applications. It is also closely connected to Android security-analysis research, where third-party code is often treated either as a source of risk or as noise that must be separated from app-specific code in malware and vulnerability studies.

4.2 Main Research Questions, Methods, and Findings in the Review Corpus

Table 4.1 summarizes the dominant primary-study question types reconstructed from the review corpus most relevant to this thesis. Rather than reproducing the review papers’ own research questions, the table captures the recurring question types that those reviews report across their underlying primary studies. The strongest support comes from the systematic reviews on Android third-party libraries.

Table 4.1: Main research questions in the underlying corpus of studies on open-source and third-party libraries in Android APKs

Primary-literature research question	Typical methods used in the primary studies	Main findings reported across the corpus
How can third-party or open-source libraries be identified inside APKs, especially when apps are obfuscated, minimized, or repackaged? (Zhan et al., 2021, 2022)	APK reverse engineering, package and class signature matching, code-similarity methods, feature matching, empirical detector construction, and benchmarking	Library identification is a major research stream, but existing approaches still struggle with obfuscation, partial imports, and version variation
How can researchers distinguish library code from app-specific code so that malware detection, clone detection, and repackaging analysis are not distorted? (Zhan et al., 2022)	Preprocessing pipelines, library filtering and removal, whitelist and non-whitelist detection, and comparative experiments with and without Third-party libraries (TPL) removal	TPLs are repeatedly treated as noise in app analysis; removing or identifying them improves downstream analyses, but whitelist methods remain limited
What security and privacy risks do embedded libraries introduce into Android apps? (Zhan et al., 2022)	Static analysis, taint and data-flow analysis, permission analysis, privacy-flow inspection, and market-scale app measurement	Primary studies consistently show that TPLs can introduce privacy leakage, security exposure, and wider attack surfaces in host apps
How do vulnerable or outdated libraries propagate risk across many apps in the Android ecosystem? (Zhan et al., 2022)	Vulnerability scanning, dependency tracing, version tracking, and large-scale APK analysis	Vulnerabilities in reused libraries can spread broadly across the mobile ecosystem when libraries remain unpatched or outdated
How can dependency conflicts and version conflicts caused by third-party libraries be detected or mitigated? (Zhan et al., 2022)	Dependency analysis, conflict detection, compatibility checking, and mitigation-framework proposals	Direct and transitive dependencies can cause runtime exceptions, crashes, and other unexpected host-app failures
How can Android apps or libraries be analyzed when they use obfuscation, packing, or anti-analysis techniques? (Dahiya et al., 2025; Sutter et al., 2024; Zhan et al., 2022)	Obfuscation-resilient matching, anti-evasion experiments, hardened instrumentation, and hybrid analysis	Obfuscation and anti-analysis remain core reasons for false negatives and incomplete coverage in both library detection and security analysis

Primary-literature research question	Typical methods used in the primary studies	Main findings reported across the corpus
Which machine-learning or deep-learning models perform best for Android malware or vulnerability detection? (Dahiya et al., 2025; Senanayake et al., 2023; Zakeya et al., 2022)	Supervised classification, deep learning, semi-supervised learning, and comparative evaluation on malware or vulnerability datasets	Many studies compare classifiers, but the reviews do not indicate one universally best model; performance varies with features, datasets, and experimental design
How can Android source-code vulnerabilities be detected as early as possible during development? (Senanayake et al., 2023)	Static code analysis, bytecode analysis, reverse engineering, machine learning on code representations, and developer-tool proposals	The reviewed corpus strongly emphasizes early detection in development workflows rather than post-release discovery only
How can vulnerability detection be integrated into developer environments and secure-development workflows? (Senanayake et al., 2023)	Tool and plugin proposals, IDE integration concepts, and workflow-oriented detection mechanisms	The literature increasingly sees detection as insufficient by itself; studies call for tools that support developers during coding and validation
What dynamic analysis techniques best reveal runtime security problems that static analysis misses? (Sutter et al., 2024)	Instrumentation, tracing, network monitoring, dynamic taint analysis, fuzzing, and automated testing	Dynamic analysis studies focus on exposing runtime behaviors such as data exfiltration, hidden execution paths, and execution-dependent malicious behavior
How can dynamic analysis achieve enough code coverage and realistic behavior triggering? (Sutter et al., 2024)	Automated UI testing, event generation, fuzzing, and emulator- and device-based experimentation	Code coverage is one of the most repeated limitations in the dynamic-analysis corpus, affecting validity and completeness of findings
How should third-party and open-source components in Android apps be understood as part of the broader software supply chain? (Reichert & Obelheiro, 2024; Williams et al., 2025)	Dependency mapping, software composition analysis, supply-chain threat modeling, and component risk analysis	The broader literature positions libraries not just as app components, but as supply-chain elements whose weaknesses can propagate downstream

Primary-literature research question	Typical methods used in the primary studies	Main findings reported across the corpus
How can the licenses of reused OSS components be identified, assessed, and managed? (Li et al., 2025)	License identification, risk assessment, mitigation and compliance workflows, and software-composition-analysis and SBOM-oriented studies	The OSS literature repeatedly asks not only what code is reused, but also what legal obligations and compatibility risks come with that reuse

4.3 Why This Topic Attracts Research Attention

The review corpus points to four main reasons why this topic has become important. First, Android applications depend heavily on third-party and open-source code. Zhan et al. (2022) note that third-party libraries represent a substantial share of Android application code, which makes their analysis a central rather than peripheral issue. Second, these libraries introduce security, privacy, and maintenance risks, including data leakage, increased attack surfaces, vulnerable dependencies, and dependency conflicts (Senanayake et al., 2023; Zhan et al., 2022). Third, the issue is now framed as part of the broader software supply chain security problem, where weaknesses in external components can propagate downstream across many systems (Reichert & Obelheiro, 2024; Williams et al., 2025). Fourth, there is a legal and organizational dimension: reused open-source components may create license-compliance obligations and compatibility risks that are difficult to identify and manage in practice (Li et al., 2025). This thesis is closest to the first and third areas, because it focuses on detecting Android library usage and making hidden software supply-chain relationships more visible.

4.4 Research Gaps Identified in Prior Reviews

The broader software supply chain literature identifies several unresolved gaps that are directly relevant to this thesis. Williams et al. (2025) argue that the component layer of the software supply chain remains insufficiently secured and that stronger mechanisms are still needed for trust, transparency, and integrity around external software artifacts. They also emphasize that open-source security is constrained by human and organizational factors, including limited resources, weak governance, and uneven security practices among maintainers and organizations (Williams et al., 2025). In a similar way, Reichert and Obelheiro (2024) highlight gaps such as limited treatment of cloud-related supply chains, relatively few proposals focused on software distribution, and insufficient use of threat-modeling frameworks in supply-chain research.

More specifically for Android, Zhan et al. (2022) report that research on Android third-party libraries still struggles with obfuscation, version identification, partial imports, and the relative lack of work on native libraries. They also observe that outdated and vulnerable libraries continue to threaten the Android ecosystem because developers often delay replacing older versions (Zhan et al., 2022). From the legal

and compliance perspective, Li et al. (2025) identify continuing difficulties in license identification, risk assessment, and risk mitigation, together with a clear gap between academic proposals and industrial uptake. Taken together, these gaps show that even when libraries can be detected technically, their provenance, security condition, and compliance implications often remain only partially understood in practice (Li et al., 2025; Williams et al., 2025; Zhan et al., 2022).

Chapter 5

Methodology

5.1 Research Design

This research follows an empirical software-engineering methodology in which a sample of real Android applications is collected, processed automatically, and analyzed to extract information about open-source library usage. The overall approach consists of three main phases: data acquisition, automated dependency extraction, and analytical evaluation of the resulting dataset. The goal is not to study a single application, but to build a repeatable pipeline that can process many APK files and generate structured information about external libraries and their characteristics.

The methodology is exploratory and data-driven. As we will see in the following sections, the study does not rely only on declared dependencies; it reconstructs actual library usage inside compiled APK files. This requires reverse engineering, pattern matching, and aggregation of results into a dataset that can be analyzed statistically.

5.2 Research Workflow

As we move through the methodology, we can follow the research workflow in three main phases: dataset preparation, automated extraction of dependency information, and analysis of the generated dataset. The detailed implementation is described later in Chapter “System Design and Pipeline”, so here we focus only on the conceptual steps. The core idea is to first identify libraries from license declarations found in the decoded APK corpus, then use the source code of those libraries to generate package-based fingerprints. These fingerprints are then matched against all decoded APKs to find evidence of library usage, including cases where a library is present in the code but not declared in the license information. This allows us to compare declared and detected dependencies in the final results.

1. Selecting a dataset of Android applications
2. Downloading and decoding APK files
3. Extracting information about open-source libraries
4. Building a reference database of library fingerprints
5. Matching fingerprints against decoded applications

6. Aggregating the results into a structured dataset
7. Extracting GitHub metadata for detected libraries and creating metrics to describe their characteristics
8. Performing statistical and structural analysis

The workflow is implemented as an automated pipeline so that the same procedure can be repeated on different samples.

5.3 APK Dataset Selection

Applications are obtained from the AndroZoo dataset (P. Liu et al., 2020), which provides large-scale collections of Android APK files (over 26 million APKs) together with metadata such as package name, hash, and market origin. This is the appropriate source for the study because it supports reproducible APK selection and automated download. Only the most recent versions of Google Play originated applications are included (almost 8.6 million APKs), and the study focuses on sensitive categories. The download order itself is not treated as a research preference; applications are processed according to availability and pipeline execution. The corpus size is limited by available hardware resources because APK decoding, license extraction, and license validation are computationally expensive. For this reason, the pipeline was first tested on 10 APKs and then scaled up iteratively to a final corpus of 700 APKs. Although AndroZoo is the main source, standalone APK files can also be added to the corpus when insights about a specific application are needed, as shown for BudapestGO in Appendix Table 1.

5.4 Dependency Extraction Approach

Because the original source code of applications is not available, the study relies on reverse-engineering techniques. APK files are decoded into an intermediate representation that allows inspection of package names, class paths, and resource files. Library detection is performed using two complementary methods: extraction of declared license information and fingerprint-based matching against known open-source repositories.

5.5 Fingerprint-Based Identification

To improve detection accuracy, a database of library fingerprints is created from public repositories. Package names extracted from repository source code are used as identifiers and compared with the decoded APK content. This approach allows detection of both declared and undeclared libraries and makes it possible to verify whether license declarations correspond to actual usage.

5.6 Data Aggregation and Analysis

For each processed application, detected libraries and related metadata are stored in structured files and combined into a single dataset. The dataset is then analyzed to

understand frequency, and differences between declared and detected dependencies. The purpose of the analysis is to obtain insights about dependency usage in real Android applications rather than to evaluate application functionality.

5.7 Repository Metadata and Health Metric

After library detection, the study enriches the dataset with GitHub repository data for detected open-source libraries. The repository-level analysis uses the health-score method proposed by Wang and Wu (2022). This method produces a single repository health score based on 12 indicators grouped into pressure, state, and response layers. The resulting score is then converted into repository health levels: low, medium, and high. These health scores and levels provide context beyond simple library presence and support the later analysis of which APKs depend on libraries whose upstream repositories are classified as low or medium health.

5.8 Methodological Limitations

The method has several limitations. The APK corpus is limited by hardware resources because the pipeline is designed to run on a local device; therefore, decoding, indexing, matching, and repository processing could not be scaled without practical runtime and storage constraints. The corpus is also not category-balanced, so the findings should not be interpreted as representing all Google Play categories equally.

The analysis is static, meaning that it detects embedded code evidence in decoded APK files but does not prove runtime execution of a library. The method also depends on successful APK decoding, available license evidence, and public repository URLs. If an APK cannot be decoded fully, or if repository information is missing from license files, some libraries cannot be linked to upstream repositories or assigned a health score.

A particularly important false-negative risk comes from code obfuscation and shrinking tools used in Android development, such as ProGuard, R8, DexGuard, or similar commercial protectors. These tools can rename packages, classes, methods, and fields; remove unused code; inline or merge classes; encrypt strings; and restructure bytecode. Because the fingerprint-matching method relies mainly on recognizable package prefixes, class paths, and import/export evidence, strong obfuscation can hide or distort the identifiers needed to match an APK class back to its original open-source library. As a result, a library may be present in the APK but remain undetected by the pipeline.

Finally `apktool` is the decoding tool used to inspect APK contents in human readable format. It adds another limitation, because some code may remain in binary form or appear with transformed identifiers, reducing the effectiveness of static fingerprint matching.

Chapter 6

System Design and Pipeline

6.1 Overview

The goal of the pipeline is to automatically detect open-source libraries used inside Android APK files and to build a reproducible dataset describing the dependency structure of real applications (Figure 6.1). The implementation follows an automated shell workflow, `workflow.sh`, which executes Python scripts at each step and connects dataset preparation, APK decoding, license extraction, repository lookup, fingerprint generation, and cross-matching into a single process (The pipeline is available on <https://github.com/sghiaseddin/Analysis-System-Open-Source-Library-in-APKs>). Because the study is experimental, CSV files are used as lightweight databases so that intermediate outputs remain human-readable and easy to import into later pipeline stages. The pipeline was developed iteratively during the research and refined to support larger samples, additional library types, and more reliable matching.

6.2 Input Data Sources

The primary source of APK files is the AndroZoo dataset, which provides large-scale collections of Android applications together with metadata such as SHA256 hash, package name, and market origin. The dataset enables reproducible selection of applications and supports automated download through an API.

In the first step, the large metadata CSV is filtered to keep only the most recent version of each application and to restrict the sample to apps available on Google Play. This filtering reduces the dataset size while preserving representative applications. Category information is then collected using a crawler to obtain official Google Play categories. The final dataset used in the pipeline contains only applications belonging to sensitive categories such as finance and healthcare.

6.3 APK Acquisition and Decoding

After the dataset is prepared, APK files are downloaded automatically using the AndroZoo API. Each APK is then decoded using the `apktool` utility, which converts the compiled application into a readable directory structure containing resources, manifest files, and smali code. Decoded applications are stored in a directory structure indexed by package name, which simplifies later lookup and cross-matching.

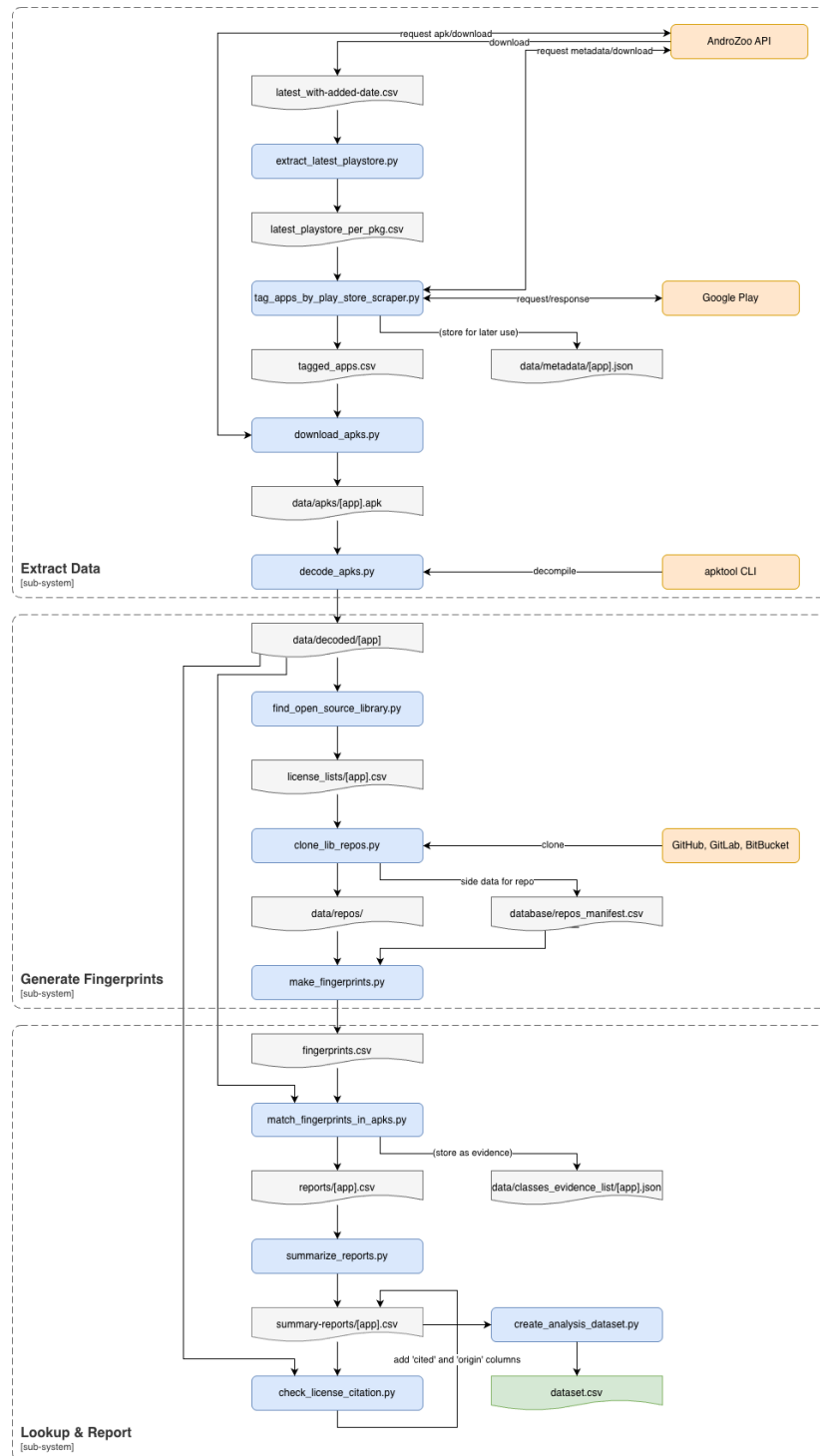


Figure 6.1: Overview of the data acquisition and analysis pipeline

6.4 License Extraction

The next step searches the decoded project tree for license declarations. The workflow scans files for keywords such as `license` and extracts structured information including library name, repository URL, license type, and evidence path. The results are stored in per-application CSV files.

This step provides an initial list of candidate libraries, but license declarations alone are not sufficient because many applications use libraries without including explicit notices. Therefore, license extraction is only the first stage of library identification and must be combined with fingerprint-based matching in later steps.

6.5 Repository Collection

Libraries that contain repository URLs pointing to GitHub, GitLab, or Bitbucket are treated as public open-source dependencies. Each unique repository is cloned locally to obtain the original source code of the library. This enables extraction of reliable identifiers from the library itself instead of relying only on declarations inside the APK.

6.6 Fingerprint Generation

For each cloned repository, the pipeline scans the source tree for package declarations in Java and Kotlin files. Package names are extracted and stored as fingerprints that uniquely identify the library (Look at the logic in the Appendix Figure 1). Additional support was later added for JavaScript libraries by detecting package declarations in distribution files, because some Android apps embed web components that are not visible through Java class matching (Look at the ratio of success for each fingerprint generation method based on final corpus analysis in the Appendix Figure 2).

All fingerprints are stored in a central CSV database containing the library name, repository URL, license information, and detected package prefixes. This database grows as more repositories are processed, increasing the probability of detecting libraries in later APK scans.

6.6.1 Illustrative Example of Open-Source Library Detection

To see how the generated fingerprints are used in practice, we can look at Glide as a simple example: a Java image-loading and bitmap-decoding library whose package namespace includes `com.bumptech.glide`. Figure 6.2 shows one example from the decoded APK `com.rangebank.grip` AKA *Range Bank*. The class `BitmapDrawableDecoder` was detected through export and import evidence. The license was not found in this APK, but the repository path for Glide was identified in another decoded APK at `data/decoded/hu.webvalto.bkkfutar/assets/licenses.json`.

```

1 .class public Lcom/bumptech/glide/load/resource/bitmap/BitmapDrawableDecoder;
2 .super Ljava/lang/Object;
3 .source "BitmapDrawableDecoder.java"
4
5 # interfaces
6 .implements Lcom/bumptech/glide/load/ResourceDecoder;
7
8 # annotations
9 .annotation system Ldalvik/annotation/Signature;
10     value = {
11         "
```

Figure 6.2: Sample evidence of Glide library usage in a decoded APK

Note. Left: export evidence at `data/decoded/com.rangebank.grip/smali_`
`classes7/com/bumptechnology/glide/load/resource/bitmap/BitmapDrawableDecoder.smali`.

Right: import evidence at `data/decoded/com.rangebank.grip/smali/androidx/core/graphics/ImageDecoderKt.smali`.

6.7 Library Matching in APK Files

The fingerprint-matching step is based on the principle that library imports and class definitions visible in the decoded codebase must originate from a defined package namespace (Figure 6.2). Therefore, when the pipeline finds an import or class reference that matches the fingerprint of a known library, this is interpreted as evidence that the library is used in the APK.

After the fingerprint database is built, the pipeline scans decoded APK files for these import and class-definition references in smali code. The extracted paths are compared against stored package fingerprints using prefix matching. This step acts as the main detection gate: once library usage is identified, the later analysis checks whether the corresponding license was declared in the APK or whether the library was detected only through code evidence.

The matching step enables detection of both declared and undeclared libraries and was optimized using binary search to reduce runtime when processing larger datasets.

6.8 Aggregation and Reporting

The final step aggregates all detected information into a unified report. For each application, the pipeline records detected libraries, license declarations, repository references, and matching evidence. The aggregated report also checks whether license notices appear close to the corresponding package usage or are stored in external assets, which helps evaluate compliance practices.

The reporting step also supports iterative improvement of the pipeline. When missing libraries are discovered, the repository list and fingerprint database can be extended and the matching step repeated without repeating the earlier stages.

The final deliverable of this process is `dataset.csv`, where each row reports one APK-library relationship together with citation status and supporting evidence for the detected library.

6.9 Computational Efficiency Techniques

Several optimizations were introduced to make the pipeline practical for larger APK corpora. The most important improvement was to avoid repeatedly scanning every decoded file during lookup. Instead, for each decoded APK, the pipeline creates a compact evidence index containing only lines relevant to library imports, exports, and comments. Each indexed entry stores the file path, the original line of code, and the extracted import or class-reference evidence. This structure reduces the search space while preserving the information needed to prove library usage.

Fingerprint matching was also optimized by sorting class and package identifiers and using *binary search* where possible. This made comparisons between decoded APK evidence and known library fingerprints more efficient than repeated linear scans. The same principle was applied in the final license-checking stage, where the pipeline must compare thousands of license records against a very large decoded corpus.

The performance gain was substantial. In the final corpus, 6,536 license records had to be checked against approximately 20 million decoded files. Initial tests showed that a direct lookup over this data could take more than 20 hours on a normal PC. After creating the evidence index and using optimized lookup techniques, the same process completed in approximately 156 minutes for the 700-APK corpus. These improvements made repeated experimentation and pipeline refinement feasible within the available hardware limits.

Chapter 7

Data Analysis

7.1 Generated Dataset

The final dataset produced by the pipeline represents aggregated evidence of open-source library usage across the analyzed APK corpus. Each row in the dataset corresponds to a detected library within a specific application, together with supporting evidence extracted from both license declarations and fingerprint matching.

The dataset contains the following key attributes:

- **pkg_name**: identifier of the Android application
- **repo_host, repo_path, repo_url**: information about the source repository of the detected library
- **library_name**: human-readable name of the library
- **library_key**: normalized identifier for grouping libraries
- **cited**: binary indicator showing whether the library was declared via license (1) or only detected through fingerprints (0)
- **origin**: method of detection (license, fingerprint, or both)
- **smali_prefix**: package prefix used for matching
- **fingerprint_types**: type of fingerprint used (e.g., Java, Kotlin)
- **classes_matched**: number of matched classes in the APK
- **sample_class, sample_class_file**: evidence of detection within the decoded APK

7.1.1 Corpus Scaling Strategy

To evaluate the robustness and scalability of the pipeline, the dataset was constructed incrementally using increasing corpus sizes. The number of analyzed APKs was expanded stepwise as follows:

[10, 20, 30, 40, 60, 80, 100, 150, 200, 300, 500, 700]

This approach allows observation of how key metrics evolve as more applications are included, and whether the results stabilize at larger scales.

7.1.2 Evaluated Metrics

The following key performance indicators (KPIs) were extracted from the dataset at each corpus size:

- Number of detected libraries per APK
- Ratio of cited vs. uncited libraries (license declared vs. undeclared)
- Number of unique libraries discovered
- Growth rate of newly discovered libraries
- Ratio of detection methods, comparing fingerprint-based evidence with license-based evidence
- Number of matches per library (classes_matched)

7.1.3 Observed Trends

The scaling analysis reveals several important patterns:

- The number of evidences of open source library use per APK increases slowly across corpus sizes, although the available data does not show the exact point at which this metric fully stabilizes (Figure 7.1).
- The number of unique libraries continues to grow with dataset size, but with diminishing returns, indicating that the most common libraries are discovered early (Figure 7.2) (Total numbers available in appendix Figure 3.)
- A significant portion of libraries are detected through fingerprint matching, suggesting that license declarations alone are incomplete in many applications (Figure 7.3).
- The ratio between cited and uncited libraries becomes relatively stable from the 300-APK corpus onward, indicating consistent behavior across larger samples (Figure 7.4).
- Larger corpora improve the coverage of the fingerprint database and can increase detection accuracy, but the current 700-APK cap does not allow a precise conclusion about the final stabilization point.

The following figures illustrate the trends observed during scaling the corpus. Figure 7.3 shows that from the 100-APK corpus onward, more than 90% of cited libraries are also detected through fingerprint matching. This supports the pipeline assumption that a license declaration and repository URL found in one APK can be used to build fingerprints from the original source repository and detect the same open-source library in other APKs.

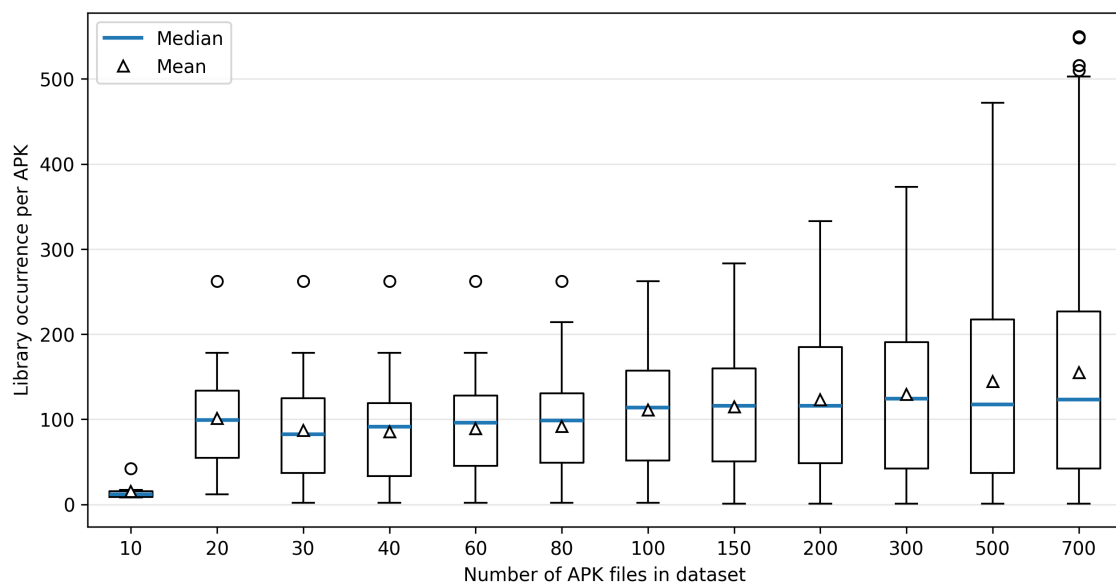


Figure 7.1: Distribution of evidence rows per APK

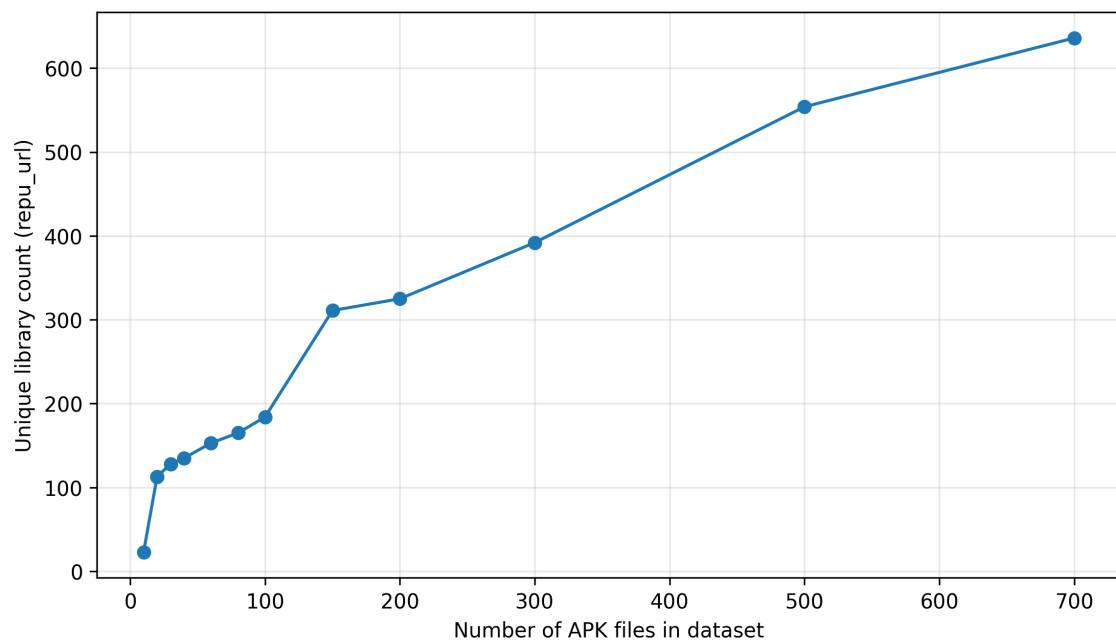


Figure 7.2: Growth of unique libraries discovered as corpus size increases

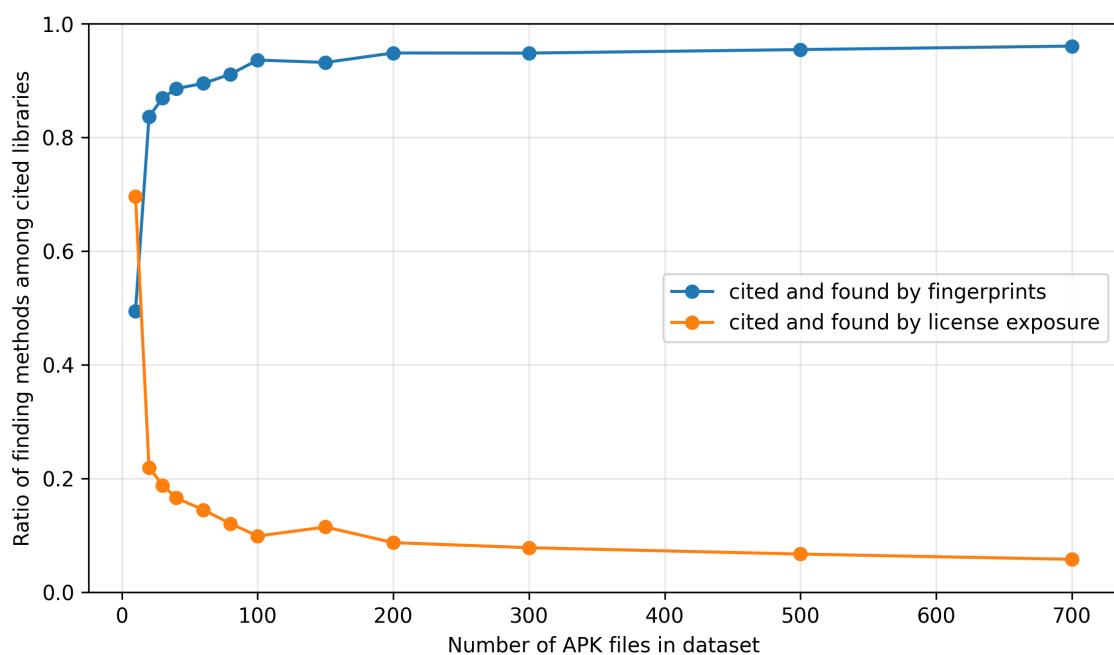


Figure 7.3: Share of cited library records also confirmed by fingerprint evidence across corpus sizes

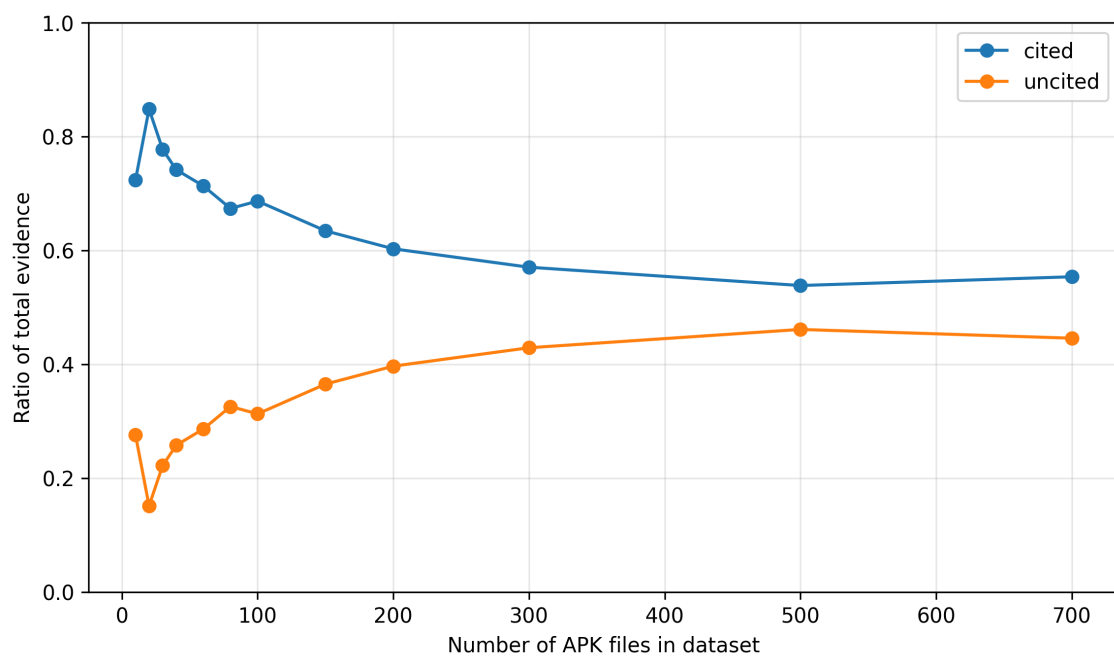


Figure 7.4: Ratio of cited vs. uncited libraries across corpus sizes

7.1.4 Final Dataset Selection

Based on the scaling analysis we saw above, and considering the hardware resource limitation, the final corpus contains 699 APKs after one APK was excluded because its lookup process exceeded the time limit (Table 7.1). As we can also see from the category counts, the sample was not balanced by category. The pipeline selected applications from the target research categories according to availability in the AndroZoo input list and download execution, without imposing a preferred category distribution.

Table 7.1: APK count by Google Play category in the final corpus

Category	APK count
FINANCE	289
HEALTH_AND_FITNESS	206
COMMUNICATION	83
MAPS_AND_NAVIGATION	65
MEDICAL	56
Total	699

7.2 GitHub-Based Repository Health Score

After analyzing APK-level metrics, we now move to repository-level evaluation using GitHub data. Prior studies show that open-source project health cannot be evaluated from a single raw repository attribute. Di Ruscio et al. (2015) present OSSMETER as a platform for automatically analysing OSS projects across multiple information sources, including source-code repositories, communication channels, and issue trackers. Their work is relevant because it frames OSS assessment as a multi-source measurement problem covering quality, maturity, development activity, and user support rather than only repository popularity.

A more direct health-score approach is proposed by (Jiang et al., 2019), who use GitHub data and factor analysis to evaluate OSS project health. Their model transforms basic repository indexes into six synthetic health-related metrics: community activity, project popularity, development activity, completeness, responsiveness, and persistence. This is important for the present thesis because it shows that repository health can be represented through combined indicators derived from project activity and community behavior.

Building on this line of work, (Wang & Wu, 2022) provide the most suitable model for this thesis because they define an explicit OSS project health score, organize the indicators into the pressure–state–response model, and report final weights for the selected factors. Therefore, this thesis adopts the Wang and Wu health-score method as the repository-level metric used for the detected GitHub libraries.

7.2.1 Adopted Health-Score Model

Wang and Wu (2022) propose a method for evaluating the health of open-source software projects based on the pressure–state–response (PSR) model. Their model divides repository health into three categories: pressure, state, and response. Across

these categories, they define 12 factors (see Table 7.2). The impact column indicates whether a larger value improves the health score (+) or reduces it (-). The weights are the final combined weights reported by Wang and Wu after combining AHP and entropy weighting. The full explanation of the Wang and Wu model, including its theoretical background and data-collection logic, will be provided in Appendix .1.

Table 7.2: Repository health factors adopted from Wang and Wu’s OSS health model

Factor title	Category	Impact	Weight
Number of watches	Pressure	+	0.0901
Number of forks	Pressure	+	0.1519
Number of languages	Pressure	-	0.2291
Number of bugs	Pressure	-	0.5285
Number of contributors	State	+	0.3147
Number of commits	State	+	0.1304
Number of comments	State	+	0.0676
Number of issues	State	+	0.4871
Number of pull requests	Response	+	0.0559
Pull requests merged	Response	+	0.3525
Issue close/open ratio	Response	+	0.4261
Issue close time	Response	-	0.1650

7.2.2 Application to Repositories Detected in the APK Corpus

I adopted Wang and Wu method for the GitHub repositories identified through the APK-library detection pipeline. Starting from the repository URLs extracted from the final APK dataset, the pipeline collects repository-level information using the GitHub API and, where necessary, local cloned repositories. The APK corpus contains 274 unique GitHub repository URLs. The health-score data collection successfully gathered the required indicators for 220 repositories, while 54 repositories were excluded or marked incomplete because of missing data, unavailable repositories, API errors, or incompatible repository structure (reflected as nan in Figure 7.6).

For each repository, the 12 factors are normalized before the layer scores are calculated. This step is necessary because the factors use different scales. For example, forks, commits, and comments are count variables, while issue close time is measured in days. Following Wang and Wu, positive factors are normalized as:

$$a'_i = \frac{a_i - \min(a)}{\max(a) - \min(a)} \quad (7.1)$$

Negative factors are normalized in the opposite direction:

$$a'_i = \frac{\max(a) - a_i}{\max(a) - \min(a)} \quad (7.2)$$

This means that higher normalized values always represent better health. For example, the minimum number of forks in the repository dataset is 0 and the maximum is 78,914, in *BoltsFramework/Bolts-Android* with 535 forks receives the normalized positive value:

$$\frac{535 - 0}{78914 - 0} = 0.0068 \quad (7.3)$$

In contrast, issue close time is treated as a negative factor, and the minimum close time is 0.01 day while the maximum is 2621 days, in *BoltsFramework/Bolts-Android* with an average close time of 55.24 days receives:

$$\frac{2621 - 55.24}{2621 - 0.01} = 0.9789 \quad (7.4)$$

After normalization, the pressure, state, and response scores are calculated as weighted sums of their respective factors. The final repository health score is then calculated as the average of the three layer scores multiplied by 100:

$$H = \left(\frac{H_P + H_S + H_R}{3} \right) \times 100 \quad (7.5)$$

7.2.3 Repository Health-Score Results

The resulting health score provides a single value for each repository, where a higher score indicates a healthier repository. Figure 7.5 shows the top 10 healthiest repositories in the APK corpus.

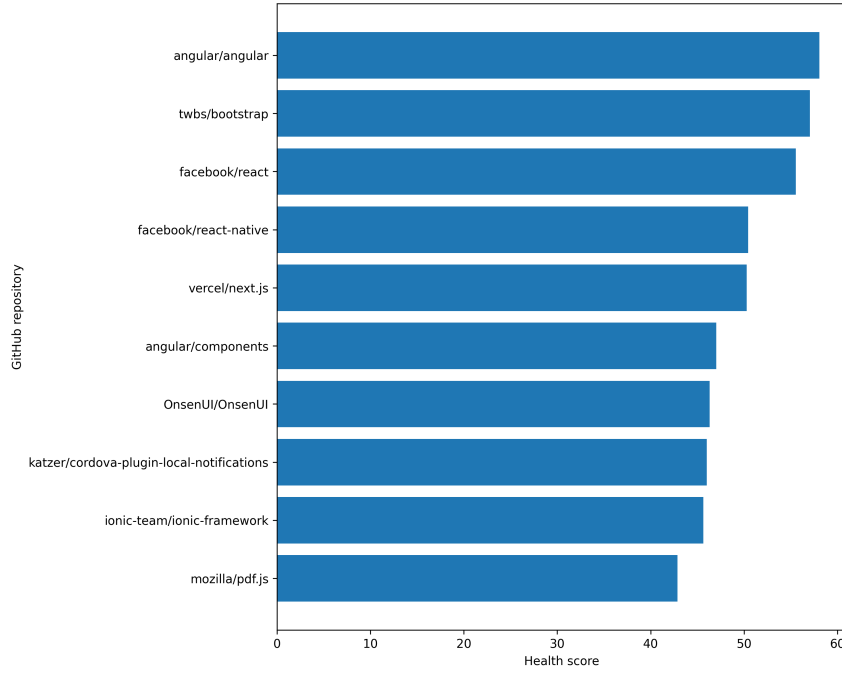


Figure 7.5: Top 10 repositories health score in the APK corpus

Wang and Wu divide repository health scores into three levels: low health in the interval $[0, 10)$, medium health in the interval $[10, 30)$, and high health in the interval $[30, 100]$. This thesis adopts the same thresholds so that the resulting repository categories remain consistent with the source model. In the analyzed GitHub repository set, no repository falls into the low-health category, 16 repositories fall into the medium-health category, and 204 repositories fall into the high-health

category. Figure 7.6 shows the distribution of repositories across these health levels in the APK corpus.

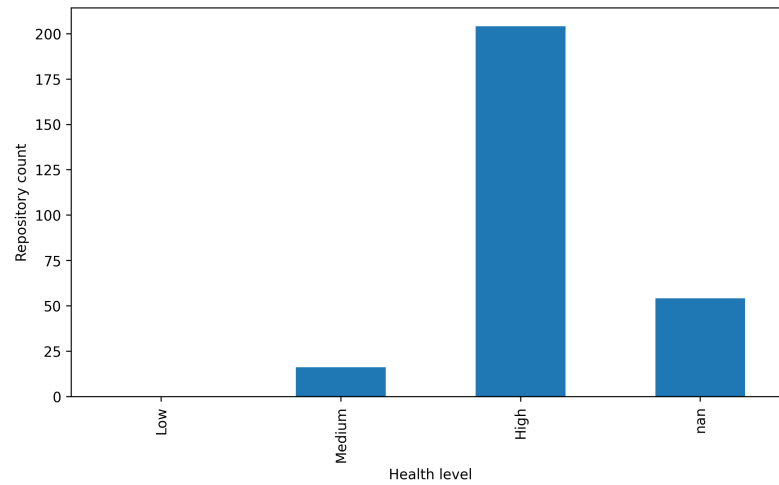


Figure 7.6: Distribution of repositories across health levels in the APK corpus

Note. The **nan** category represents repositories that were excluded or marked incomplete because of missing data, unavailable repositories, GitHub API errors, or incompatible repository structure.

The repository-level health data will later be joined back to the APK-library evidence dataset. This makes it possible for us to estimate which APKs depend on medium-health libraries and how intense those libraries appear in each APK. The APK-level interpretation of this joined dataset is discussed later in Subsection 9.1.3.

Chapter 8

Validation

8.1 Validation Strategy

At this point, the pipeline has identified open-source libraries used in each analyzed APK through license extraction and fingerprint matching. The main validation question is whether these detected libraries can be compared against an independent baseline. Initially, the planned validation approach was to inspect applications manually and compare the pipeline output with the list of open-source libraries disclosed inside the application itself. This would have provided a direct user-visible validation source if applications exposed their dependency declarations through an in-app license screen.

However, this first validation strategy was not successful as a general baseline. A sample of 27 commercial applications (Appendix Table 2) was installed and explored manually in the Android Studio Emulator, including settings menus, about pages, help pages, and other sections where open-source notices are typically located. Almost none of the selected applications provided an accessible in-app list of open-source libraries. The only clear exception was *BudapestGO*, where the in-app license screen matched the libraries detected by the pipeline (Appendix Table 1). This result was useful as a qualitative example, but it was not sufficient for estimating accuracy, precision, recall, or false positives across the pipeline.

For this reason, a second validation approach was introduced using open-source Android applications from F-Droid. The motivation was to create a control set where the APK can be analyzed by the fingerprinting pipeline, while the corresponding public source-code repository can be inspected independently. In this setting, the source-code analysis is treated as an approximate ground truth because dependencies may be declared in Gradle files, version catalogs, third-party documentation, vendor directories, or repository references. The validation therefore compares two independent sources of evidence for the same applications: libraries detected from the compiled APK and libraries reconstructed from the public source code.

8.2 Open-Source Control Set

The validation control set consists of 100 F-Droid applications. These applications were selected from the F-Droid dataset under three conditions. First, the package name had to exist in the main APK input dataset (AndroZoo). Second, the application had to belong to one of the research categories used in the main corpus. Third,

the F-Droid metadata had to contain a GitHub source-code URL. This ensured that each selected application had both an APK representation for pipeline analysis and a source-code repository for independent comparison.

The APK analysis pipeline was then executed separately for these 100 F-Droid applications. For this validation run, the fingerprint database was not limited to fingerprints generated from the F-Droid repositories alone. Instead, it combined the fingerprints already gathered from the main 699-APK corpus with the additional fingerprints generated from the 100 F-Droid APKs and their related source-code evidence. This design made the validation run closer to the real pipeline setting, because the APKs were analyzed against the accumulated fingerprint knowledge available in the project rather than against an isolated minimal fingerprint set.

The resulting APK-side validation dataset contains **15,353 evidence rows** across exactly **100 APK packages**. These rows correspond to **202 unique open-source libraries** when grouped by `repo_url`, and none of the rows has a missing repository URL. This means that the validation subset is suitable for repository-level comparison because all evidences can be linked to a normalized repository identifier. The average number of evidence rows of open source library use is **153.53 per APK** (Figure 8.1) and the average number of distinct repository URL of open source libraries is **13.63 per APK**, both are very close to the main 699-APK corpus average (we will see in Chapter 9).

The citation pattern in the validation subset also shows that license disclosure remains incomplete even among open-source applications: approximately **68.55%** of evidences of open source library use are cited, while **31.45%** are not cited. Among the cited rows, approximately **95.90%** are also found by fingerprint evidence, while **38.13%** are found through license-exposure evidence. This indicates that even in the F-Droid validation subset, fingerprint-based evidence remains the dominant confirmation mechanism, while direct license exposure captures only part of the dependency evidence.

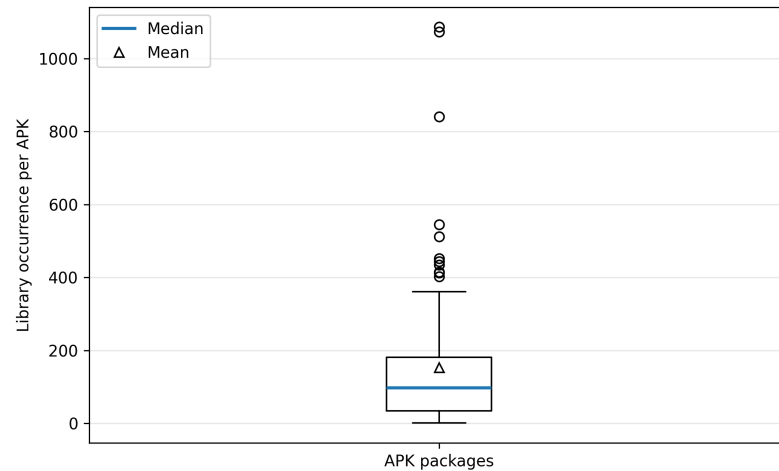


Figure 8.1: Distribution of detected open source library use per APK in the F-Droid validation set

8.3 Source-Code Library Extraction Methods

The source-code validation step uses multiple detector methods. Each method scans the cloned repository in a different way and contributes candidate library declarations. At this stage, five source-code detection methods are used:

1. **Gradle dependency detection:** scans `build.gradle` and `build.gradle.kts` files for dependency declarations such as `implementation`, `api`, `compileOnly`, `runtimeOnly`, `kapt`, and test-related configurations.
2. **Version catalog detection:** scans `libs.versions.toml` files and extracts entries from the `[libraries]` section.
3. **Third-party markdown detection:** scans documentation files in directories such as `thirdparty`, `third_party`, and `vendor`, including cases where libraries are listed in markdown notes (README and LICENSE).
4. **Vendor directory detection:** scans common bundled-library directories such as `thirdparty`, `third_party`, `vendor`, `external`, and `libs`, and records subdirectories as possible embedded libraries.
5. **GitHub URL detection:** scans text-like files across the repository for GitHub repository URLs and records them as candidate third-party libraries. Links pointing to the application repository itself are excluded so that internal project references are not counted as external libraries.

8.4 Matching APK Findings Against Source-Code Findings

The matching process uses `pkg_name` as the first-level identifier. This means that a library detected in an APK is only compared with source-code libraries extracted from the corresponding application's repository. Within each package, the second-level matching uses multiple identifiers: `library_key`, `library_name`, Gradle coordinates, repository path, repository URL, and extracted GitHub URLs.

Because the APK-derived dataset and source-code-derived dataset do not always describe the same library in the same format, exact matching alone would be too strict. Therefore, the comparison uses several partial matching methods:

1. **Repository URL matching:** compares normalized repository URLs and repository paths, including partial matches such as `owner/repo` appearing inside a longer URL.
2. **Namespace matching:** checks whether one normalized library key or library name appears inside the other. For example, `okhttp` may match `com.squareup.okhttp3:okhttp`.
3. **Artifact matching:** extracts the artifact part from Gradle coordinates or repository paths and compares this shorter identifier.

8.5 Validation Results on the F-Droid Control Set

The validation produced the following matching results for the 100 F-Droid applications:

- Matched source-code rows: **2,741**
- Matched APK rows: **11,740**
- Total match relationships: **106,197**

The match relationships were distributed across the matching methods as follows:

Table 8.1: Distribution of match types in the F-Droid validation set

Match type	Number of match relationships
Namespace match	63,810
Artifact match	28,108
Repository URL match	14,279
Total	106,197

Using the source-code analysis as the approximate ground truth, the APK analysis achieved the following validation metrics:

Table 8.2: Validation metrics using source-code analysis as approximate ground truth

Metric	Value
True positive matched source rows	2,741
False negative source rows missed by APK analysis	3,738
False positive APK rows not supported by source analysis	3,613
Recall / ground-truth coverage	42.31%
Precision / APK supported by source	43.14%
F1 score	42.72%

These results show that the validation baseline is more informative than the original manual in-app emulator inspection. The recall value indicates that the APK pipeline identifies a meaningful portion of source-code-derived dependency evidence, but also misses a substantial number of libraries visible in the source-code analysis. The precision value shows that many APK-detected rows are not directly confirmed by the current source-code extraction method. This does not automatically mean that all of these rows are false positives. Source-code dependency extraction is itself incomplete and may miss generated dependencies, bundled code, build-time transformations, or libraries introduced through external build logic. In addition, part of the 3,613 APK rows not confirmed by source-code evidence may represent transitive dependencies present in the compiled APK rather than explicitly declared direct dependencies in the scanned source-code files.

8.6 Limitations of Validation

The validation should be interpreted as an approximate baseline rather than a perfect ground truth. Source code provides stronger evidence than in-app license screens, but

static source-code scanning may still miss conditional dependencies, build variants, aliases, generated code, bundled binaries, and dependencies introduced through external build logic. Conversely, the compiled APK may contain libraries that are not clearly visible in the scanned source branch, including transitive dependencies or build-time additions.

The matching logic is intentionally permissive because APK evidence and source-code declarations often use different identifiers. Partial matching improves coverage, but it can also introduce false matches. A more exact validation would require opening each project in Android Studio and manually inspecting the full build configuration and codebase. Increasing the corpus size and fingerprint database may also change the accuracy in both directions: it can improve detection by discovering more libraries, but it can also introduce more transitive APK libraries that are difficult to match back to explicit source-code declarations. Therefore, the validation results should be read as a practical performance baseline for the current pipeline, not as a final measurement of absolute correctness.

Chapter 9

Results and Discussion

9.1 Findings

In this section, we will see the empirical findings derived from the APK-level analysis, focusing on library detection, citation behavior, and dependency structure. The results are based on the output dataset generated by the pipeline for the 699 APK corpus size.

9.1.1 Summary of Key Performance Indicators (KPIs)

The analysis of the APK corpus reveals consistent patterns in open-source library usage. The following KPIs summarize the main characteristics of the dataset:

- Average number of libraries per APK: **154.84**
- Average number of repositories per APK: **15.34**
- Total number of unique libraries across corpus: **636**
- Ratio of cited libraries: **55.39%**
- Ratio of uncited libraries: **44.61%**
- Ratio of library records supported by fingerprint-based evidence: **96.06%**
- Total evidence rows of open source library use: **108,234**

These indicators suggest that Android applications rely heavily on external libraries, with an average of 154.84 evidence rows of open source library use per APK and the average number of 15.34 distinct repository URL of open source libraries per APK. to compare, the OSSRA report states that the average audited application contained 1,180 open-source components, with 64% of components introduced transitively (Duck, 2026). This comparison should be interpreted cautiously because the OSSRA dataset is not necessarily Android-specific and may use a different counting method.

At the same time, only 55.39% of detected libraries are explicitly declared through license information, while 44.61% are identified exclusively through fingerprint-based detection. This demonstrates a substantial gap between actual dependency usage and documented license disclosure.

These indicators suggest that Android applications rely heavily on external libraries, with an average of 154.84 evidences per APK. At the same time, only 55.39% of detected libraries are explicitly declared through license information, while 44.61% are identified exclusively through fingerprint-based detection. This demonstrates a substantial gap between actual dependency usage and documented license disclosure.

The high fingerprint-based evidence ratio (96.06%) shows that the fingerprint method delivers the expected result and can identify most library usage through code evidence. This is also computationally important because fingerprint lookup is less costly than full license-exposure lookup. The license-exposure check is therefore used mainly in the final stage to revalidate whether detected libraries are actually cited, and the results show that many are not. Overall, 48,280 of 108,234 evidences (44.61%) were found only through fingerprint matching, showing that license declarations are frequently incomplete or absent, with no clear category-level pattern (Figure 9.1). The detection-origin distribution for the 699 APK corpus is shown in Appendix Figure 4 (Also look at the full summary at Appendix Table 3).

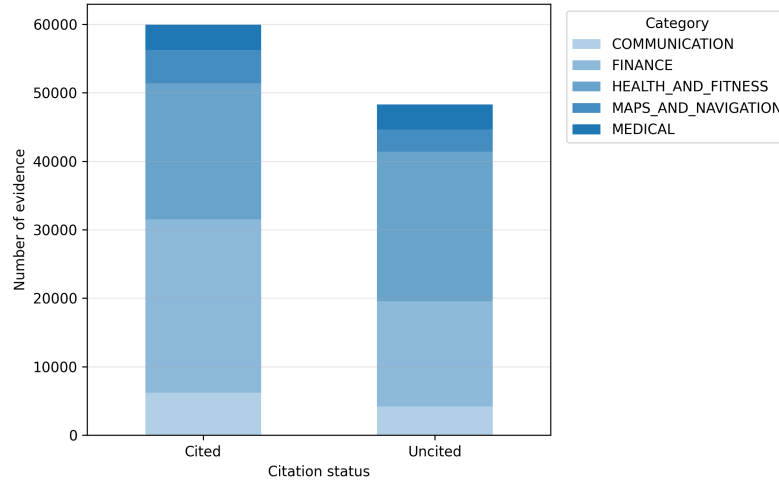


Figure 9.1: Distribution of cited versus uncited libraries

9.1.2 Distribution of Libraries Across APKs

The distribution of libraries per application shows high variability. As shown in Figure 9.2, the mean number of evidences of open source library use per APK is 154.84, while the median is lower at 123, indicating that some applications contain much larger dependency sets. In the final corpus, 55 APKs have fewer than 10 detected libraries, whereas 52 APKs have more than 400 detected libraries (For detailed info look at Appendix Figure 6 and Figure 4.)

The APK-level results show that dependency usage in Android applications is both extensive and partially hidden.

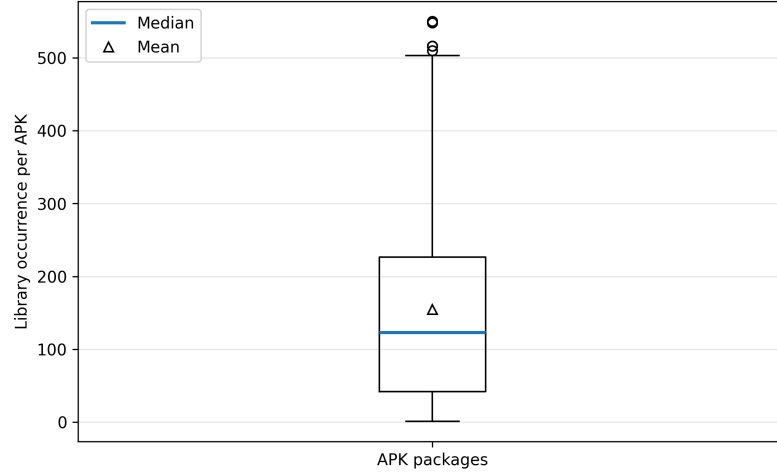


Figure 9.2: Distribution of evidences of open source library use per APK

9.1.3 Medium-Health Level Open Source Library Usage in APKs

After the APK-level analysis, the repository-level Wang and Wu health scores were joined back to the APK/library evidence dataset. This makes it possible to identify which applications use libraries classified as having *medium* repository health and to measure how intensely those libraries appear inside each APK. In this analysis, usage intensity is measured through `classes_matched`, which is used as a proxy for how strongly a detected library appears in the decoded APK codebase.

Across the largest analyzed corpus, the dataset contains **108,234** APK/library evidence rows. Of these, **98,330** rows could be joined to a GitHub repository with an assigned health level. Within this joined dataset, **225** rows correspond to libraries classified as *medium health*. These medium-health rows appear at least one time in **89** distinct APKs. This means that medium-health libraries are not limited to isolated cases; they appear across a **12.7%** of the analyzed application corpus size.

How intense is medium-health library use inside each APK?

The intensity of medium-health library usage is calculated as the share of class matches associated with medium-health libraries relative to all detected open-source class matches in the same APK:

$$MediumHealthIntensity = \frac{medium_health_classes_matched}{apk_total_classes_matched} \times 100 \quad (9.1)$$

This measure does not claim to represent runtime execution frequency. Instead, it captures how much of the detected open-source code evidence inside an APK is associated with repositories classified as medium health. Applications with high values should therefore be interpreted as having stronger structural exposure to medium-health libraries.

Which APKs use open-source libraries with medium health?

The first question is which APKs contain at least one open-source library classified as *medium health*. The analysis identifies **89** such APKs. For each APK, the aggregation records the number of medium-health evidence rows, the number of matched classes associated with these libraries, and the percentage of the APK's total detected open-source class matches represented by medium-health libraries.

The ranking is based on intensity of medium-health library use (`medium_health_intensity_pct_of_apk`). The highest observed values include *KidiCom Chat™ (FR)* at **18.75%**, *CEMAH netCEM* at **1.71%**, and *NeurOm* at **0.57%** (Figure 9.3). These applications therefore show the strongest relative presence of medium-health libraries within their detected open-source class usage.

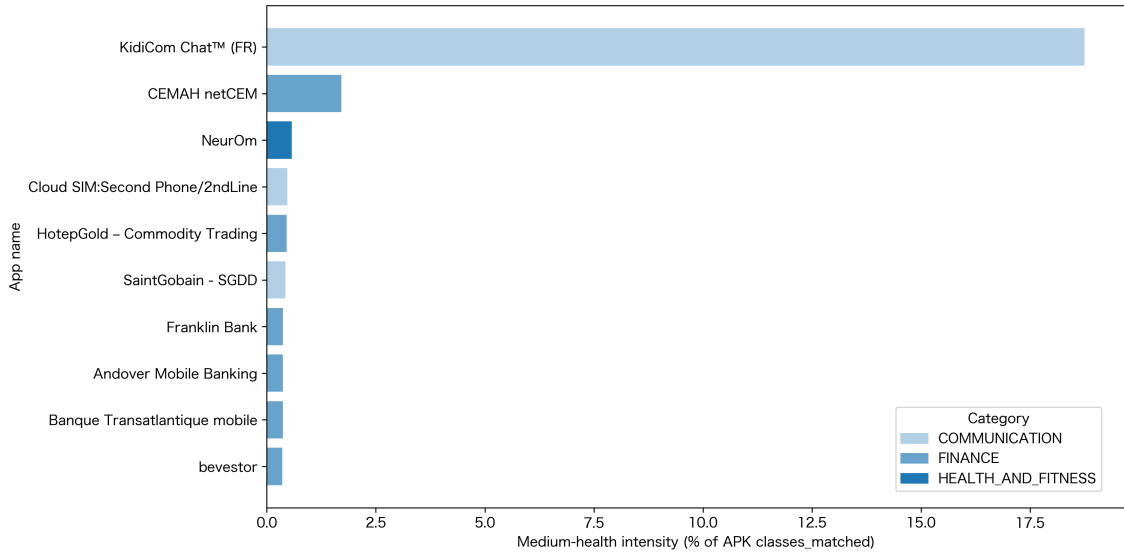


Figure 9.3: Top 10 APKs by medium-health library usage intensity

Which APKs use medium-health libraries but uncited (cited = 0)?

The second part of the analysis focuses only on medium-health rows where `cited = 0`. These are cases where the pipeline found code evidence of a medium-health library, but no corresponding license declaration was identified in the APK evidence. The dataset contains **69** uncited medium-health rows, appearing across **34** distinct APKs.

The leading APKs by uncited medium-health library use intensity include *HotepGold – Commodity Trading*, *bevestor*, and *Dubai Bus on Demand* (Figure 9.4). These cases are important because the relevant medium-health libraries are detected through code evidence but are not visible through license citation. As a result, even a technically skilled user would have little practical opportunity to identify these libraries from the APK disclosure information alone and react if one of them became risky.

9.2 Interpretation of Results

From these findings, we can draw the following main interpretations:

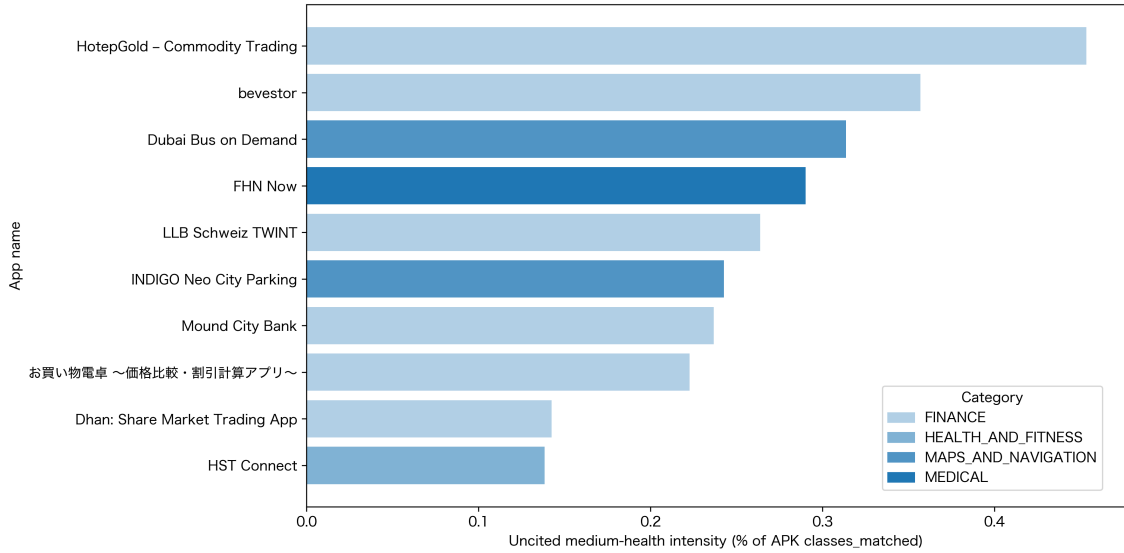


Figure 9.4: Top 10 APKs by uncited medium-health library usage intensity

- Android applications in the corpus rely heavily on external open-source libraries, with an average of 154.84 evidences of open source library use per APK.
- License disclosure is incomplete: 44.61% of evidences of open source library use are uncited, meaning they are found through code evidence but not through license declarations.
- The fingerprint-based method is effective for this research goal, because 96.06% of cited library records are also confirmed by fingerprint evidence.
- Dependency usage varies widely across applications, from APKs with 1 to 550 evidences of open source library use.
- Repository-level analysis shows that, among the 220 repositories with assigned Wang and Wu health levels, 204 are classified as high health and 16 as medium health, while no repository falls into the low-health category.
- Medium-health libraries appear in 225 evidence rows across 89 distinct APKs, representing 12.7% of the analyzed application corpus.
- Uncited medium-health libraries are especially important because users and auditors cannot easily identify them from license disclosure alone; in this corpus, 69 uncited medium-health evidence rows appear across 34 APKs.

Overall, the results show that the main issue is not only the presence of open-source libraries, but the gap between actual library usage and what is disclosed to users or reviewers. The repository-health analysis adds another layer to this finding by showing which detected libraries come from repositories with lower health levels.

9.3 Implications for Developers

The findings show that many libraries are present in APK code but are not visible through license declarations. Developers should therefore make dependency visibility

and license declaration part of the CI/CD process, not a final manual documentation step. Software composition analysis, SBOM-style documentation, fingerprint-based checks, and repository-health monitoring can help teams verify which libraries are actually embedded in their applications and reduce the gap between used and declared dependencies.

9.4 Implications for End Users

As end users, we face limited transparency. Because many libraries are uncited, we cannot realistically identify which dependencies an app contains, whether they are secure, or whether they are actively maintained. This is especially important for apps in sensitive categories such as finance, health, communication, and navigation. Since we have almost no practical way to evaluate hidden dependencies, uninstalling unused apps in these categories is a reasonable risk-reduction practice.

Chapter 10

Conclusion

10.1 Answer to Research Questions

We can now return to the research questions and answer them based on the results. The thesis investigated whether external open-source libraries can be detected in compiled Android APK files and whether the detected dependencies are consistently disclosed through license information. The results show that fingerprint-based evidence can reveal substantial library usage that is not visible from license declarations alone.

RQ1: Is it feasible to detect external libraries in Android APK files? Yes. The developed pipeline successfully processed the APK corpus, decoded applications, extracted license evidence, generated fingerprints from source repositories, and matched those fingerprints against decoded APK code. In the final corpus, the pipeline produced 108,234 evidence records across 699 APKs, showing that automated library detection is feasible at this scale.

RQ2: How reliable are license declarations and package fingerprints for identifying library dependencies? License declarations are useful but incomplete. In the final corpus, only 55.39% of evidences of open source library use were cited, while 44.61% were uncited and visible only through code evidence. Package fingerprints were more effective for detection: 96.06% of cited library records were also confirmed by fingerprint evidence. This supports the use of fingerprints as the main detection method, with license exposure used mainly for citation validation.

RQ3: How many open source libraries, and on what health level, are used in a sample of Android applications? The final corpus contains 636 unique repository URLs and an average of 154.84 evidences of open source library use per APK. The libraries are mainly detected through Java and Kotlin package fingerprints. Repository-level analysis also showed that among the 220 GitHub repositories with assigned health levels, 204 repositories were classified as high health, 16 repositories were classified as medium health, and no repository was classified as low health.

RQ4: Which Android applications show the highest dependency on low-health or medium-health open-source libraries? The analysis identified 225 medium-health evidence rows across 89 distinct APKs and it doesn't identify any low-health libraries. Dependency strength was measured by medium-health library use intensity, meaning the percentage of an APK's total detected open-source class matches that belong to medium-health repositories. Based on this intensity measure, the leading applications are *KidiCom Chat™ (FR)*, *CEMAH netCEM*, and *NeurOm*.

These applications therefore have the strongest relative exposure to medium-health libraries within their detected open-source code evidence.

Overall, the practical part of the thesis was applicable and completed: the APK collection was built, the decoding and matching pipeline was implemented, GitHub metadata was collected, repository health scores were calculated, and the final dataset was analyzed. The pipeline can also target a standalone APK, add it to the corpus, identify its open-source library ingredients, assess their repository health levels, and compare it with other APKs. The validation chapter showed that in-app license screens are usually not a reliable validation baseline, while the F-Droid source-code control set provided a more useful approximate ground truth and confirmed that the pipeline can recover a meaningful portion of dependency evidence, although both precision and recall remain limited by differences between compiled APK contents and source-code declarations.

Some expectations were only partially achieved. The corpus size remained limited by hardware and lookup time, the sample was not category-balanced, manual validation was limited because most applications did not expose in-app license information, and the method cannot prove runtime execution of a library. Therefore, the results should be interpreted as evidence of embedded library presence, repository-health context, and disclosure gaps, not as a complete runtime security assessment.

10.2 Future Work

The pipeline created a labeled corpus that records which files and lines contain evidence of external library imports, exports, and class references. This creates an opportunity for future work based on text analysis and machine learning. Instead of relying only on rule-based fingerprint matching and repeated lookup operations, future versions of the system could tokenize the indexed evidence lines and train a model to classify whether a line is likely to indicate external library usage. Such a model could use features from package names, import patterns, file paths, comments, and surrounding code context.

This direction is realistic because prior research has shown that source-code tokenization and machine-learning representations can support software engineering and cybersecurity tasks. However, it should be treated as a follow-up improvement rather than a replacement for the current method. The current fingerprint approach is interpretable and produces direct evidence for each library match, while a machine-learning model would need careful validation to avoid false positives.

Another future improvement is to refactor the pipeline for a cloud environment such as AWS. This would reduce the current hardware limitation, allow the analysis of a much larger APK corpus, and support parallel decoding, repository cloning, fingerprint generation, and matching. A larger corpus would also enrich the fingerprint database, which could increase the chance of finding additional evidence of open-source library use in APKs.

References

- Ajila, S. A., & Wu, D. (2007). Empirical study of the effects of open source adoption on software development economics. *Journal of Systems and Software*, 80(9), 1517–1529. <https://doi.org/10.1016/j.jss.2007.01.011>
- Boldi, P., & Gousios, G. (2020). Fine-Grained Network Analysis for Modern Software Ecosystems. *ACM Trans. Internet Technol.*, 21(1), 1:1–1:14. <https://doi.org/10.1145/3418209>
- Chesbrough, H. (2023, March). *Measuring the Economic Value of Open Source: A Survey and a Preliminary Analysis* (tech. rep.). The Linux Foundation. <https://doi.org/10.70828/MNOD1451>
- Cordey, S. (2023, January). *Software Supply Chain Attacks: An Illustrated Typological Review* (tech. rep.). ETH Zurich. <https://doi.org/10.3929/ETHZ-B-000584947>
- Cusumano, M. (1989). The Software Factory: A Historical Interpretation. *IEEE Software*, 6(2). Retrieved January 18, 2026, from <https://www.proquest.com/openview/0072a929a6eb814d538e735728aa65ed/1?pq-origsite=gscholar&cbl=37787>
- CVE-2022-22965 Analyzing the Exploitation of Spring4Shell Vulnerability in Weaponizing and Executing the Mirai Botnet Malware.* (2022, April 8). Trend Micro. Retrieved January 24, 2026, from https://www.trendmicro.com/en_us/research/22/d/cve-2022-22965-analyzing-the-exploitation-of-spring4shell-vulner.html
- Dahiya, A., Singh, S., & Shrivastava, G. (2025). Android malware analysis and detection: A systematic review. *Expert Systems*, 42(1), e13488. <https://doi.org/10.1111/exsy.13488>
- del Águila, I. M., del Sagrado, J., & Cañadas, J. (2020, February). Software Engineering Timeline: Major areas of interest and multidisciplinary trends. <https://doi.org/10.48550/arXiv.2002.10163>
- Di Ruscio, D., Kolovos, D. S., Korkontzelos, I., Matragkas, N., & Vinju, J. J. (2015). OSSMETER: A software measurement platform for automatically analysing open source software projects. *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 970–973. <https://doi.org/10.1145/2786805.2803186>
- Duck, B. (2026). *Open Source Security and Risk Analysis (OSSRA) Report* (Corporate Report). Black Duck Software, Inc. <https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html>
- Düsing, J., & Hermann, B. (2022). Analyzing the Direct and Transitive Impact of Vulnerabilities onto Different Artifact Repositories. *Digital Threats*, 3(4), 38:1–38:25. <https://doi.org/10.1145/3472811>
- Fitzgerald, B. (2006). The Transformation of Open Source Software1. *MIS Quarterly*, 30(3), 587–598. <https://doi.org/10.2307/25148740>

- Freire, R. (2024). *Understanding Red Hat's response to the XZ security incident*. Red Hat Blog. Retrieved January 24, 2026, from <https://www.redhat.com/en/blog/understanding-red-hats-response-xz-security-incident>
- Hoffmann, M., Nagle, F., & Zhou, Y. (2024). The Value of Open Source Software. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4693148>
- Jiang, S., Cao, J., & Prasad, M. (2019). The Metrics to Evaluate the Health Status of OSS Projects Based on Factor Analysis. In Y. Sun, T. Lu, Z. Yu, H. Fan, & L. Gao (Eds.), *Computer Supported Cooperative Work and Social Computing* (pp. 723–737). Springer. https://doi.org/10.1007/978-981-15-1377-0_56
- Kumar, S. H. B. I., Sampaio, L. R., Martin, A., Brito, A., & Fetzer, C. (2024). A Comprehensive Study on the Impact of Vulnerable Dependencies on Open-Source Software. *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, 96–107. <https://doi.org/10.1109/ISSRE62328.2024.00020>
- Lerner, J., & Tirole, J. (2002). Some Simple Economics of Open Source. *The Journal of Industrial Economics*, 50(2), 197–234. <https://doi.org/10.1111/1467-6451.00174>
- Lerner, J., & Tirole, J. (2005). The Economics of Technology Sharing: Open Source and Beyond. *Journal of Economic Perspectives*, 19(2), 99–120. <https://doi.org/10.1257/0895330054048678>
- Li, B., Liu, C., Fan, L., Chen, S., Zhang, Z., & Liu, Z. (2025). Open Source, Hidden Costs: A Systematic Literature Review on OSS License Management. *IEEE Transactions on Software Engineering*, 51(9), 2432–2454. <https://doi.org/10.1109/TSE.2025.3586411>
- Libraries.io - security & maintenance data for open source software*. (2026, March 22). Libraries.io. Retrieved March 22, 2026, from <https://libraries.io>
- Liu, C., Chen, S., Fan, L., Chen, B., Liu, Y., & Peng, X. (2022). Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem. *Proceedings of the 44th International Conference on Software Engineering*, 672–684. <https://doi.org/10.1145/3510003.3510142>
- Liu, P., Li, L., Zhao, Y., Sun, X., & Grundy, J. (2020). AndroZooOpen: Collecting Large-scale Open Source Android Apps for the Research Community. *MSR '20: 17th International Conference on Mining Software Repositories*, 548–552. <https://doi.org/10.1145/3379597.3387503>
- Newman, S. (2019, November). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. "O'Reilly Media, Inc."
- Npm Blog Archive: Details about the event-stream incident*. (2018). npm Blog. Retrieved January 24, 2026, from <https://blog.npmjs.org/post/180565383195/details-about-the-event-stream-incident>
- Ojha, A. K., & Rao, R. A. (2014). The Emergence of an Organizational Field: The Case of Open Source Software. *Vikalpa: The Journal for Decision Makers*, 39(2), 127–143. <https://doi.org/10.1177/0256090920140212>
- Open Source Malware Index Q3 2025: High-Severity Attacks Surge*. (2025, October 15). Retrieved January 24, 2026, from <https://www.sonatype.com/blog/open-source-malware-index-q3-2025>
- O'Reilly, T. (1999). Lessons from open-source software development. *Commun. ACM*, 42(4), 32–37. <https://doi.org/10.1145/299157.299164>

- Ponta, S. E., Plate, H., & Sabetta, A. (2020). Detection, assessment and mitigation of vulnerabilities in open source dependencies. *Empirical Software Engineering*, 25(5), 3175–3215. <https://doi.org/10.1007/s10664-020-09830-x>
- Randell, B. (1979). Software engineering in 1968. *ICSE '79: Proceedings of the 4th International Conference on Software Engineering*. <https://dl.acm.org/doi/10.5555/800091.802915>
- Reichert, B. M., & Obelheiro, R. R. (2024). Software supply chain security: A systematic literature review. *International Journal of Computers and Applications*, 46(10), 853–867. <https://doi.org/10.1080/1206212X.2024.2390978>
- Senanayake, J., Kalutarage, H., Al-Kadri, M. O., Petrovski, A., & Piras, L. (2023). Android Source Code Vulnerability Detection: A Systematic Literature Review. *ACM Comput. Surv.*, 55(9), 187:1–187:37. <https://doi.org/10.1145/3556974>
- Snyk. (2024). *2024 State of Open Source Security* (Corporate Report). Snyk Limited. <https://snyk.io/lp/state-of-open-source-2024/>
- Sonatype. (2026). *2026 State of the Software Supply Chain* (Corporate Report). Sonatype Inc. <https://www.sonatype.com/state-of-the-software-supply-chain/>
- Soto-Valero, C., Harrand, N., Monperrus, M., & Baudry, B. (2021). A comprehensive study of bloated dependencies in the Maven ecosystem. *Empirical Software Engineering*, 26(3), 45. <https://doi.org/10.1007/s10664-020-09914-8>
- Sutter, T., Kehrler, T., Rennhard, M., Tellenbach, B., & Klein, J. (2024). Dynamic Security Analysis on Android: A Systematic Literature Review. *IEEE Access*, 12, 57261–57287. <https://doi.org/10.1109/ACCESS.2024.3390612>
- Vulnerabilities Statistics 2025: Record CVE Surge*. (2025, October 8). DeepStrike. Retrieved January 24, 2026, from <https://deepstrike.io/blog/vulnerability-statistics-2025>
- Wang, H., & Wu, Z. (2022). A Method to Evaluate the Health of OSS Projects Based on the PSR Model and Combination Weighting. *2022 IEEE 2nd International Conference on Software Engineering and Artificial Intelligence (SEAI)*, 109–118. <https://doi.org/10.1109/SEAI55746.2022.9832291>
- Weber, S. (2000). The Political Economy of Open Source Software.
- Williams, L., Benedetti, G., Hamer, S., Paramitha, R., Rahman, I., Tamanna, M., Tystahl, G., Zahan, N., Morrison, P., Acar, Y., Cukier, M., Kästner, C., Kapravelos, A., Wermke, D., & Enck, W. (2025). Research Directions in Software Supply Chain Security. *ACM Trans. Softw. Eng. Methodol.*, 34(5), 146:1–146:38. <https://doi.org/10.1145/3714464>
- Yan, T., Deng, Q., Zhang, H., Fu, Y., Grunzweig, J., Harbison, M., & Falcone, R. (2021, December 10). *Another Apache Log4j Vulnerability Is Actively Exploited in the Wild (CVE-2021-44228)*. Unit 42. Retrieved January 24, 2026, from <https://unit42.paloaltonetworks.com/apache-log4j-vulnerability-cve-2021-44228/>
- Zakeya, N., Séglá, K., Chamseddine, T., & Alvine, B. B. (2022). Probing AndroVul dataset for studies on Android malware classification. *Journal of King Saud University - Computer and Information Sciences*, 34(9), 6883–6894. <https://doi.org/10.1016/j.jksuci.2021.08.033>
- Zerouali, A., Mens, T., Decan, A., & De Roover, C. (2022). On the impact of security vulnerabilities in the npm and RubyGems dependency networks. *Empirical Software Engineering*, 27(5), 107. <https://doi.org/10.1007/s10664-022-10154-1>

- Zhan, X., Fan, L., Liu, T., Chen, S., Li, L., Wang, H., Xu, Y., Luo, X., & Liu, Y. (2021). Automated third-party library detection for Android applications: Are we there yet? *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 919–930. <https://doi.org/10.1145/3324884.3416582>
- Zhan, X., Liu, T., Fan, L., Li, L., Chen, S., Luo, X., & Liu, Y. (2022). Research on Third-Party Libraries in Android Apps: A Taxonomy and Systematic Literature Review. *IEEE Transactions on Software Engineering*, 48(10), 4181–4213. <https://doi.org/10.1109/TSE.2021.3114381>

Appendix

.1 Wang and Wu OSS Repository Health Model

This appendix summarizes only the research method of Wang and Wu (2022), because the main analysis chapter adopts their repository health-score framework.

.1.1 Methodological Structure

Wang and Wu evaluate open-source software project health using the pressure–state–response (PSR) model. In their adaptation, the pressure layer represents external pressure on the project, the state layer represents the current condition of the project, and the response layer represents the project’s ability to react to problems or attacks. Each layer contains a group of repository indicators, and the final health score is calculated from the weighted values of these indicators. The model uses 12 indicators in total, grouped into the three PSR layers. The indicators and their final weights are listed in Table 7.2 in the main analysis chapter.

.1.2 Data and Preprocessing

In the original study, Wang and Wu use GitHub project data collected through GHTorrent. Their empirical design relies on repository activity, issue, pull-request, contributor, and communication-related records. Because these indicators use different units and scales, the authors first transform them into comparable normalized values. Positive indicators are normalized so that larger raw values produce larger normalized values. Negative indicators are normalized in the opposite direction so that larger raw values produce smaller normalized values.

.1.3 Weighting Method

A key methodological contribution of Wang and Wu is their combination weighting approach. They combine a subjective weighting method with an objective weighting method. The subjective part uses the analytic hierarchy process (AHP), where indicators are compared within the same PSR layer to express their relative importance. The objective part uses the entropy method, where indicators receive weight according to the amount of information and variation they contribute in the observed data.

The final indicator weights are calculated as a linear combination of the AHP weights and the entropy weights:

$$w = \mu w_{AHP} + (1 - \mu) w_{Entropy} \quad (1)$$

where w is the final indicator weight, w_{AHP} is the subjective AHP weight, $w_{Entropy}$ is the objective entropy weight, and μ controls the balance between the two components. In this thesis, the final published weights reported by Wang and Wu are adopted directly, instead of recalculating new weights from the APK corpus.

.1.4 Health-Score Calculation

After normalization and weighting, each PSR layer is calculated as a weighted sum of its indicators:

$$H_i = \sum_{j=1}^m w_j \times p_{ij} \quad (2)$$

where H_i is the layer score, w_j is the weight of indicator j , and p_{ij} is the normalized value of that indicator. The final repository health score is then calculated by combining the pressure, state, and response layer scores:

$$H = \left(\frac{H_P + H_S + H_R}{3} \right) \times 100 \quad (3)$$

This produces one repository-level health score.

.2 Extra Figures and Tables

Supporting figures for pipeline and validation. The following figures are supplementary materials referenced in the previous chapters.

```

1  #!/usr/bin/env python3
2  import argparse
3  import csv
4  import os
5  import re
6  import sys
7  from pathlib import Path
8  from datetime import datetime
9  from concurrent.futures import ThreadPoolExecutor, as_completed
10
11 # ----- Logging -----
12 def log(msg: str, *, flush=True):
13     print(f"[{datetime.now().strftime('%H:%M:%S')}] {msg}", flush=flush)
14
15 # ----- Regexes -----
16 JAVA_PKG_RE = re.compile(r'^\s*package\s*([a-zA-Z_][\w.]*?)\s*;\s*$')
17 KT_PKG_RE = re.compile(r'^\s*package\s*([a-zA-Z_][\w.]*?)\s*;\s*$')
18 MANIFEST_PKG_RE = re.compile(r'^package\s*=\s*"([^\"]+)"')
19 # maven pom.xml tags (very lightweight)
20 XML_TAG_RE = re.compile(r'<([a-zA-Z0-9_-]+)>(.*)</[a-zA-Z0-9_-]+>', re.S)
21 # gradle group definitions (kts and groovy)
22 GRADLE_GROUP_ASSIGN = re.compile(r'^\s*group\s*=\s*"([^\"]+)"\s*;$')
23 GRADLE_GROUP_STR = re.compile(r'^\s*group\s*=\s*"([^\"]+)"\s*;$')
24
25 # ----- Helpers -----
26 def to_smali_prefix(java_pkg: str) -> str:
27     """com.example.lib -> Lcom/example/lib/"""
28     java_pkg = java_pkg.strip().strip(",")
29     if not java_pkg:
30         return ""
31     return "L" + java_pkg.replace(".", "/") + "/"
32

```

Figure 1: The regular expressions to capture fingerprints in various languages such as Java (code in python)

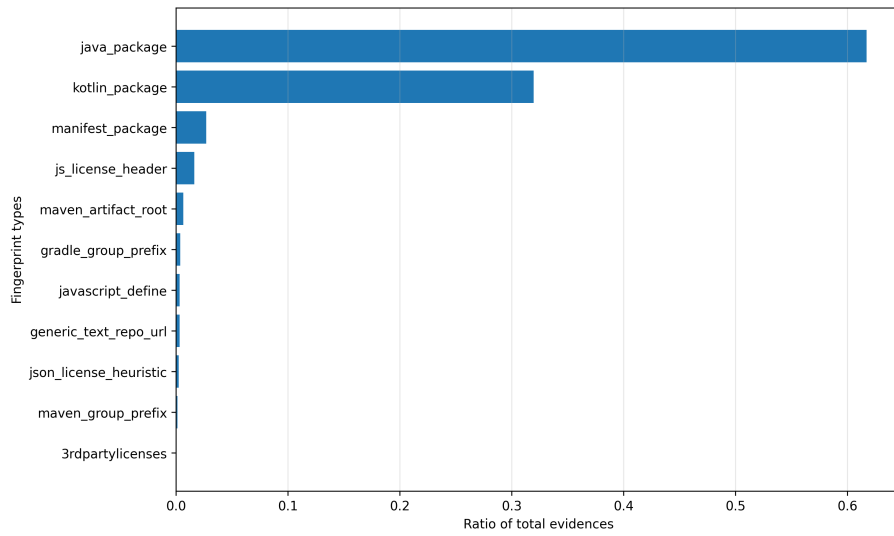


Figure 2: Distribution of fingerprint types used to detect library evidence by application category in the final APK corpus

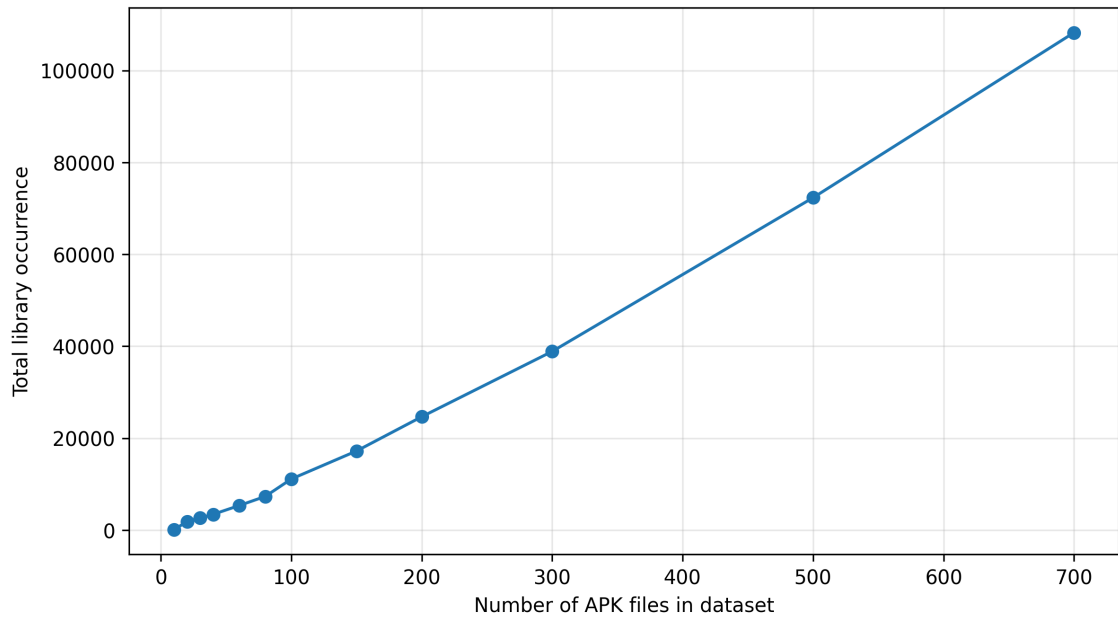


Figure 3: Total evidences of open source library use as the APK corpus size increases

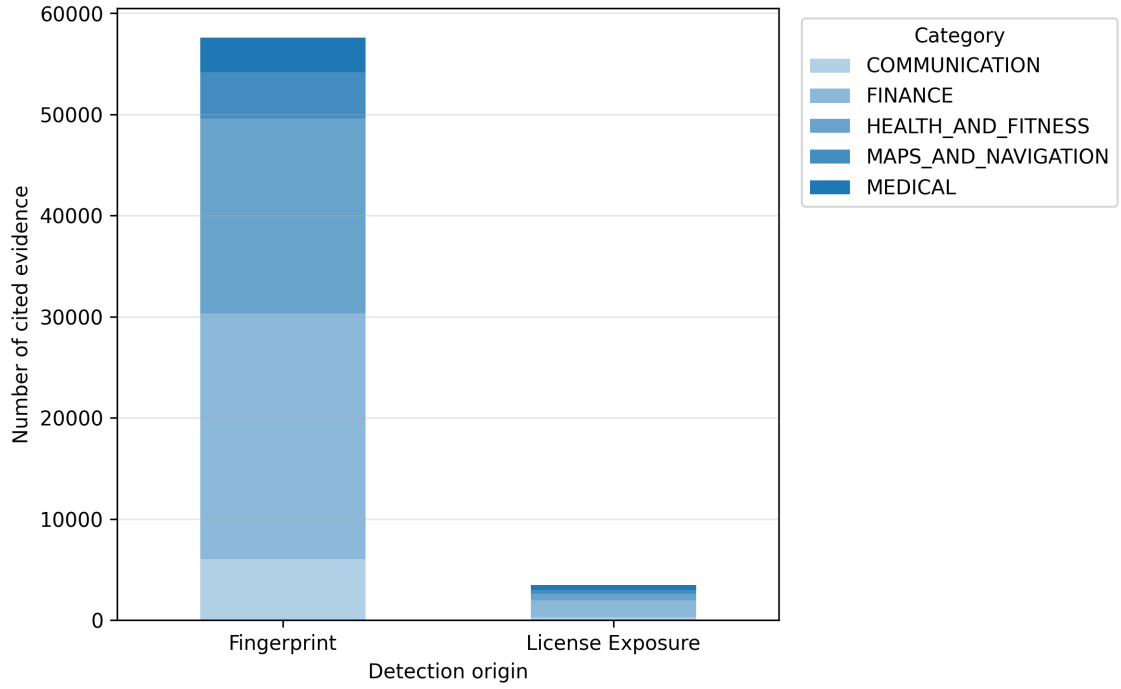


Figure 4: Detection origin of cited library records, comparing fingerprint evidence and license-exposure evidence by application category

Table 1: Summary report for the BudapestGO app as a standalone APK added to the corpus

Metric	Value
Total evidences	266
Total unique evidences	91
Cited evidences	255
Uncited evidences	11
Cited ratio	95.86%
Uncited ratio	4.35%
Evidences by fingerprint	68
Evidences by license-exposure	251
Evidences by fingerprint and license-exposure	53
Cited evidences found by fingerprints	57
Medium health evidences	0

Note. Evidence: a record on analysis dataset associated with open source library use

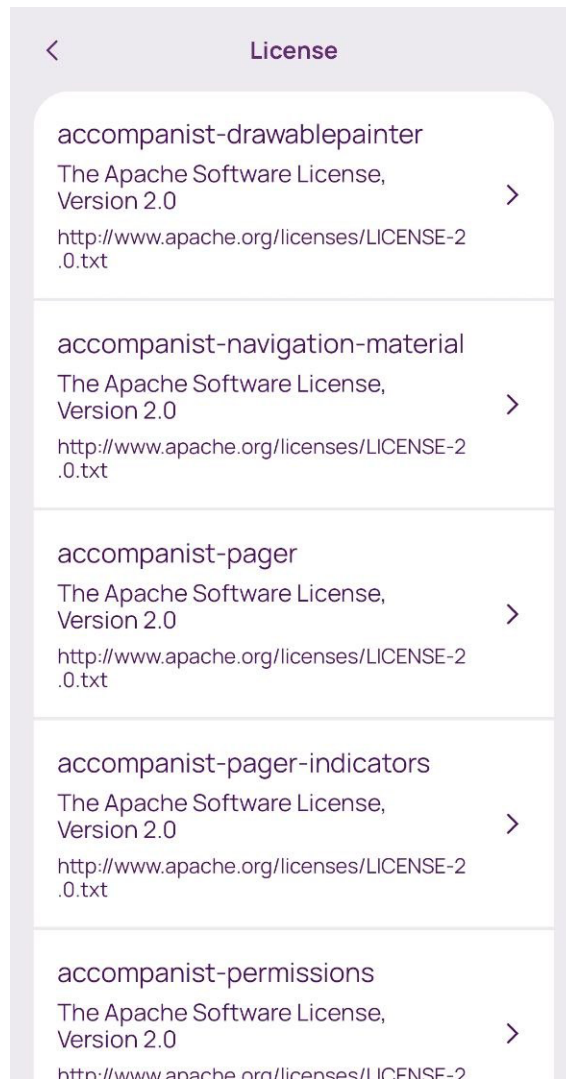


Figure 5: BudapestGO in-app license screen generated through Sentry, a helper tool used in the CI/CD stack to audit runtime external libraries and export their licenses. Screenshot address: Menu, Information and help, License.

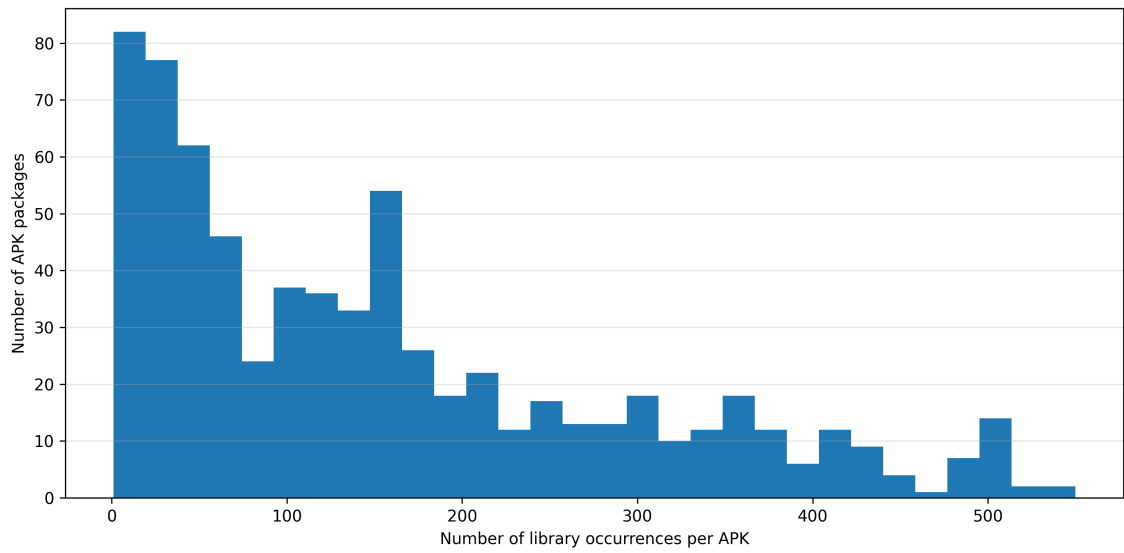


Figure 6: Histogram of evidences of open source library use per APK in the final corpus

Table 2: List of apps for manual license investigation.

app_name	downloads	category
NoiseFit: Health & Fitness	50M+	HEALTH_AND_FITNESS
Bobble AI Keyboard Memes, Gifs	50M+	COMMUNICATION
GoPay: Transfer, Payment, QRIS	50M+	FINANCE
Taxsee Driver	10M+	MAPS_AND_NAVIGATION
BSI Mobile	10M+	FINANCE
Dhan: Share Market Trading App	10M+	FINANCE
MEXC: Buy Bitcoin BTC & Crypto	10M+	FINANCE
Taptap Send: Money Transfer	10M+	FINANCE
Boost Visual Voicemail	10M+	COMMUNICATION
Agribank Plus	10M+	FINANCE
CoinMarketCap: Crypto Tracker	10M+	FINANCE
CHECK24 Vergleiche	10M+	FINANCE
Upstox Stocks IPO Mutual Funds	10M+	FINANCE
Planet Fitness	10M+	HEALTH_AND_FITNESS
Cashi	10M+	FINANCE
ABA Mobile	10M+	FINANCE
Luno: Buy Bitcoin & Crypto	10M+	FINANCE
Monzo - Mobile Banking	10M+	FINANCE
T-money GO - Taxi, Express Intercity	10M+	MAPS_AND_NAVIGATION
imo beta -video calls and chat	100M+	COMMUNICATION
Yandex Maps and Navigator	100M+	MAPS_AND_NAVIGATION
BudapestGO	1M+	MAPS_AND_NAVIGATION
in.kontakt	1K+	COMMUNICATION
Cabrillo Credit Union	10K+	FINANCE
UK Credit Union	10K+	FINANCE
BrightStar CU Mobile	10K+	FINANCE
Apple Bank Mobile Banking	100K+	FINANCE

Note. Manual license investigation was not successful; None of these apps provide in-app list of open-source libraries except *BudapestGO*.

Table 3: Merged summary KPIs for the 699 APK corpus; each dataset row represents one evidence of open source library use

Metric	Value
Total evidences	108,234
Unique APK packages	699
Evidences with missing repository URL	0
Total generated fingerprints	6,536
Cited evidences	59,954
Uncited evidences	48,280
Cited ratio of total evidences	55.39%
Uncited ratio of total evidences	44.61%
Evidences found by fingerprint	105,874
Evidences found by license-exposure	3,478
Cited evidences found by fingerprints	57,594
Fingerprint ratio among cited evidences	96.06%
License-exposure ratio among cited evidences	5.80%
Average evidences per APK	154.84
Median evidences per APK	123
Minimum evidences per APK	1
Maximum evidences per APK	550
Average repositories per APK	15.34
Median repositories per APK	15
Minimum repositories per APK	1
Maximum repositories per APK	91
Unique repository URLs	636*
Unique cloned GitHub repository attempts	274*
GitHub repositories found	220
GitHub repositories not found or returned error	54

Note. The number of unique repository URLs is higher than the number of cloned GitHub repository attempts because some repositories contain more than one library, and some license URLs point to a specific library or subdirectory inside a repository rather than to a separate repository.